



Extending Kubebuilder for Horizontal Kubernetes Operator Scaling

BACHELORARBEIT (T3_3300)

as part of the examination for

Bachelor of Science (B.Sc.)

in Computer Science

at the Baden-Württemberg Cooperative State University Karlsruhe

by

Marvin Fuchs

Submission Date 11.09.2026

Processing Period 12 Weeks

Matriculation Number, Course 8505123, TINF23B2

Training Company SAP SE
Walldorf

Department SAP BTP ABAP Environment -
Landscape Infrastructure

Supervisor at Training Company Tom Romeo Doil

Supervisor at University Dr.-Ing. Lars Briem

ABSTRACT

- Deutsch -

Mit *Kubebuilder* entwickelte *Kubernetes-operators* sind durch ein Single-Leader-Design beschränkt. Dadurch wachsen *CPU*- und Speicherverbrauch sowie die Wiederherstellungszeit nach dem Absturz des Leaders linear mit der Anzahl der verwalteten Ressourcen. Bestehende Ansätze zur horizontalen Skalierung schließen die dadurch entstehende Lücke nur teilweise: Keine der fünf in dieser Arbeit untersuchten repräsentativen Lösungen bietet gleichzeitig *minimal disruption*, *monotonicity*, gegenseitigen Ausschluss, geringe Last auf dem *API* Server und einfache *Kubebuilder*-Integration.

Diese Arbeit stellt *kube-sharder* vor, eine Drop-in-Erweiterung für *Kubebuilder*, die diese Lücke schließt. Arbeit wird mittels *Highest Random Weight (HRW)*-Consistent-Hashing auf die Instanzen verteilt, gegenseitiger Ausschluss während *membership churn* wird durch eine Lease-basierte Revision-Control-Zustandsmaschine sichergestellt, und der Speicherverbrauch kann zusätzlich durch einen optionalen sharded Cache reduziert werden. Die gesamte Koordination erfolgt ausschließlich über den *Kubernetes-API* Server, sodass außer dem sharded *operator* selbst keine externe Komponente benötigt wird. Bestehende *reconcilers* funktionieren unverändert über eine Builder-API, welche jene der *controller-runtime* widerspiegelt.

Die Bibliothek wird anhand von sieben Zielen nach dem *GQM*-Ansatz an einem synthetischen Website-*operator* evaluiert. Bei praxisrelevanten *fleet*-Größen und Ressourcenanzahlen überschreitet der Jain-Fairness-Index 0.95. Mit aktiviertem sharded Cache sinken Speicher- und *API*-Server-Request-Overhead gegenüber der ungeshardeten Baseline auf nahezu null, sobald die Ressourcenanzahl in die Tausende wächst, und die Failover-Zeit wird um einen Faktor entsprechend der *fleet*-Größe reduziert. Verglichen mit der nächstverwandten Vorarbeit erreicht *kube-sharder* dasselbe Speicher- und Failover-Verhalten ohne dediziertes externes *sharding*-Deployment und ohne zusätzliche Schreibanfragen pro übertragener Ressource.

ABSTRACT

- English -

Kubernetes operators built with Kubebuilder are bounded by a single-leader design. As a result, CPU and memory footprint, as well as recovery time after a leader crash, grow linearly with the number of managed resources. Existing horizontal-scaling approaches close parts of the resulting gap, but none of the five representative solutions surveyed in this work simultaneously provides minimal disruption, monotonicity, mutual exclusion, low API Server load, and simple Kubebuilder integration.

This thesis presents *kube-sharder*, a drop-in Kubebuilder extension that closes this gap. Work is distributed across replicas using HRW consistent hashing, mutual exclusion during membership churn is provided by a lease-based revision control state machine, and memory consumption can additionally be reduced by an optional sharded cache. All coordination is performed exclusively through the Kubernetes API Server, so no external component beyond the sharded operator itself is required. Existing reconcilers work without modification through a builder API that mirrors the controller-runtime.

The library is evaluated against seven objectives following the GQM approach on a synthetic website operator. At fleet sizes and resource counts of practical interest, the Jain fairness index exceeds 0.95. With the sharded cache enabled, memory and API Server request overhead relative to the unsharded baseline drop close to zero as the resource count grows into the thousands, and failover time is reduced by a factor equal to the fleet size. Compared to the closest prior work, *kube-sharder* reaches the same memory and failover behavior without a dedicated external sharding deployment and without additional write requests per transitioned resource.

Table of Contents

1 Introduction	1
2 Fundamentals	3
2.1 Consistent Hashing	3
2.1.1 Karger Consistent Hashing	4
2.1.2 Highest Random Weight	5
2.1.3 DxHash	7
2.2 Distributed Mutual Exclusion	10
2.2.1 Fencing tokens	10
2.2.2 Generational clock	11
2.2.3 State machine replication	12
2.3 Kubernetes	13
2.3.1 Resource Model	13
2.3.2 Application Programming Interface	16
2.3.3 API Server and etcd	19
2.4 Kubebuilder	21
2.4.1 Architecture	21
2.4.2 Manager	23
2.4.3 Client	23
2.4.4 Cache	24
2.4.5 Controller	26
3 Existing Solutions	28
3.1 Static Sharding	28
3.2 KubeVela	28
3.3 Tim Ebert's Master Thesis	29
3.4 ArgoCD	31
3.5 multicluster-runtime	32
3.6 Comparison	33
4 Methodology and Approach	35
4.1 Requirements and Environmental Constraints	37
4.1.1 Functional Requirements	38
4.1.2 Non-Functional Requirements	39
4.2 Evaluation Objectives	39
5 Concept	45
5.1 Architecture	45
5.2 Membership Discovery	47
5.3 Monitored Workqueue	48
5.4 Sharded Cache	49
5.4.1 Sharding	50
5.4.2 Resyncing	52

5.5 Revision Control	53
5.5.1 Problem	53
5.5.2 Design Space	54
5.5.3 Revision Version	54
5.5.4 State Machine	55
5.5.5 Notation	55
5.5.6 Transitions	56
5.5.7 Correctness	58
6 Evaluation	61
7 Conclusion	70
Bibliography	a
Abbreviations	g
Glossary	h
List of Figures	k
List of Tables	m
List of Code	n
Statutory Declaration	o

1 Introduction

The *operator* pattern is an integral part of the extensibility of the *Kubernetes Resource Model (KRM)*. Rather than encoding operational knowledge in static configuration, an *operator* is a piece of software that continuously observes the desired state of *Custom Resources (CRs)* and reconciles the actual state to match. This model is flexible enough to manage everything from databases and message brokers to entire application platforms.

To simplify development, frameworks like *Kubebuilder* can be used. *Kubebuilder* is a widely used official framework developed by a *Kubernetes Special Interest Group (SIG)* and built on the official *controller-runtime* library. Under the hood, a running *operator* subscribes to a *watch* stream for the resources it manages, receives events from the *Kubernetes API* Server, and enqueues them into a workqueue that dispatches reconciles to a pool of worker goroutines. To prevent two replicas from acting on the same resource, *operator* deployments rely on leader election: only one replica is *active* at any given time, while all others remain passive followers ready to take over after a failure [1].

Two consequences of this design become problematic as the number of managed resources grows. First, with thousands of resources under reconciliation, a single leader can saturate its *worker node*: *CPU* and memory both scale with the number of managed resources, and because only one replica is active at a time, available resources are bound by a single *worker node*. Additional replicas do not distribute load, they remain idle as followers. Second, *operator* failover time degrades with the same growth. When the leader crashes and a follower is elected, *controller-runtime*'s informer sync enqueues an event for every existing object, so the new leader must reconcile every managed resource once before it catches up.

The problem is recognized in the *Kubernetes* ecosystem, and multiple horizontal-scaling approaches for *operators* already exist. *Section 3* surveys five representative solutions, ranging from purely manual *sharding* to fully autonomous designs with per-resource mutual exclusion. As summarized in *Table III*, none covers the full set of properties expected from a general-purpose solution: they either require a dedicated external *sharding* component, forfeit *monotonicity*, allow concurrent reconciles during

membership churn, or greatly increase *API* Server load. Beyond these functional gaps, no existing approach integrates transparently with *Kubebuilder*, the framework used by many contemporary *Kubernetes operators* [1].

This thesis presents *kube-sharder*, a library that closes those gaps. The library extends *Kubebuilder* with support for horizontal scaling of *operators* and is designed to require only minimal code changes in existing *operators*: reconcilers work without modification, and integration only changes the *operator*'s setup code. Work is distributed across replicas via consistent hashing, with *minimal disruption* during *membership churn*, *monotonicity*, and mutual exclusion, and without an external *sharding* component. The detailed functional and non-functional requirements are formalized in *Section 4.1*.

The work is guided by two research questions:

RQ 1 Design: How can horizontal scaling be added to *operators* built with *Kubebuilder* such that existing reconcilers work unmodified, work is fairly distributed across replicas under *membership churn* with *minimal disruption* and mutual exclusion, and no external component beyond the sharded *operator* itself is required?

RQ 2 Evaluation: What overhead does the proposed library introduce compared to an unsharded *Kubebuilder operator*, and how do throughput, failover time, and resource footprint scale with the number of *shard* replicas?

The scope of this work is limited to *operators* that follow *Kubebuilder* best practices. The library targets *operators* reconciling thousands of resources, where the single-leader bound becomes a practical limitation.

The remainder of this thesis is structured as follows. *Section 2* introduces the foundations of distributed-systems and *Kubernetes* required to reason about horizontal scaling of *operators*, covering consistent hashing, distributed mutual exclusion, the *Kubernetes API* model, and the architecture of *Kubebuilder*. *Section 3* surveys the existing horizontal-scaling approaches summarized above and identifies the gaps this thesis addresses. The methodology chapter derives the functional and non-functional requirements (*Section 4.1*) and the evaluation objectives (*Section 4.2*) that the design must satisfy. *Section 5* presents the design of the proposed library and its implementation as a *Kubebuilder* extension. *Section 6* evaluates the implementation against the requirements and evaluation objectives. *Section 7* concludes the thesis and outlines future work.

2 Fundamentals

This chapter explains concepts and algorithms relevant to this thesis. First, it introduces two foundational distributed-systems topics: *sharding* assignments via consistent hashing and distributed mutual exclusion. Both are essential for understanding the proposed concept and the related work discussed in this chapter and *Section 3*. It then explains *Kubernetes* with a focus on its *API* and Resource Model, because horizontally scaling *Kubernetes operators* requires detailed knowledge of *Kubernetes* behavior under load. Finally, it details the architecture and design of *Kubebuilder*, because the horizontal-scaling solution is implemented as a *Kubebuilder* extension.

2.1 Consistent Hashing

Consistent hashing belongs to the problem domain of *sharding*. *Sharding* describes the process of assigning a key set K to a node set N through a *sharding function* $f : K \rightarrow N$. The simplest such function combines a hash of the key with the number of nodes.

Given an enumeration of nodes $N = \{n_0, n_1, \dots, n_{|N|-1}\}$ and a hash function $h : K \rightarrow \mathbb{N}$, modulo sharding is defined as

$$f(k) = n_{h(k)}; \quad h(k) = \text{mod}(|N|) \quad (1)$$

While trivial to implement, this approach reassigns nearly all keys of K whenever N changes, which greatly reduces performance when N changes [2], [3, p. 201].

Consistent hashing is the class of *sharding* algorithms that avoids this behavior by achieving *minimal disruption* under *membership churn*. *Membership churn* refers to changes of N caused by nodes joining, leaving, or being replaced, and *minimal disruption* refers to the property that the number of keys whose assignment changes stays close to the information-theoretic minimum $\frac{|K|}{|N|}$ [2]. Because consistent hashing is deterministic, it allows local, coordination-free calculation of the previously described *sharding function* as long as all nodes know about all other nodes. This chapter explains three consistent hashing algorithms to provide a broad overview of different approaches. Consistent hashing is the basis of many existing solutions described in

Section 3 and of the library introduced by this thesis. Section 5 refers to this section when selecting a suitable algorithm for the proposed solution.

2.1.1 Karger Consistent Hashing

The term *consistent hashing* was first introduced by Karger et al. in 1997 [2] with a protocol designed for caching in distributed networks. The work describes an environment in which *clients* learn about a list of *cache buckets* and need to determine which bucket is responsible for a key.

Due to clients learning about the available cache buckets independently, they hold different *views* $\mathcal{V} \subseteq \mathcal{B}$ of the full bucket set $\mathcal{B}$. The assignment function $f : 2^{\mathcal{B}} \times K \rightarrow \mathcal{B}$ is known by all clients and describes how keys are assigned to buckets for every possible view. This allows clients to calculate locally, based only on their known buckets, which bucket should hold a key.

Using a hash function that distributes keys uniformly, Karger et al.'s consistent hashing achieves the following two properties with *high probability* [2]:

Balance Each bucket of a view $\mathcal{V}$ is assigned a fraction of $O\left(\frac{1}{|\mathcal{V}|}\right)$ keys.

Monotonicity For any two views $\mathcal{V}_1 \subseteq \mathcal{V}_2 \subseteq \mathcal{B}$, the implication in (2) holds. From an initial view $\mathcal{V}_1$ to a view $\mathcal{V}_2$ with additional buckets, keys previously assigned to an old bucket may only be reassigned to a new bucket, never to a different old bucket. Monotonicity achieves *minimal disruption* in consistent hashing as defined by Karger et al.

$$\begin{aligned} \mathcal{V}_1 &\subseteq \mathcal{V}_2 \subseteq \mathcal{B} \\ f(\mathcal{V}_2, k) \in \mathcal{V}_1 &\Rightarrow f(\mathcal{V}_1, k) = f(\mathcal{V}_2, k) \end{aligned} \tag{2}$$

Figure 1 shows an example visualization of Karger et al.'s consistent hashing. Four nodes $n_1 \dots n_4$ with two buckets each are distributed across a hash space of $2^n - 1$. Six keys $k_1 \dots k_6$ are placed on the hash space and assigned to the next clockwise node. Inside the ring, the assignment for a view $\mathcal{V} = \mathcal{B}$ is shown.

Due to the $O(\log|N|)$ balance defined above, the number of buckets greatly influences how uniformly keys are distributed. If set N is small, each node must be assigned

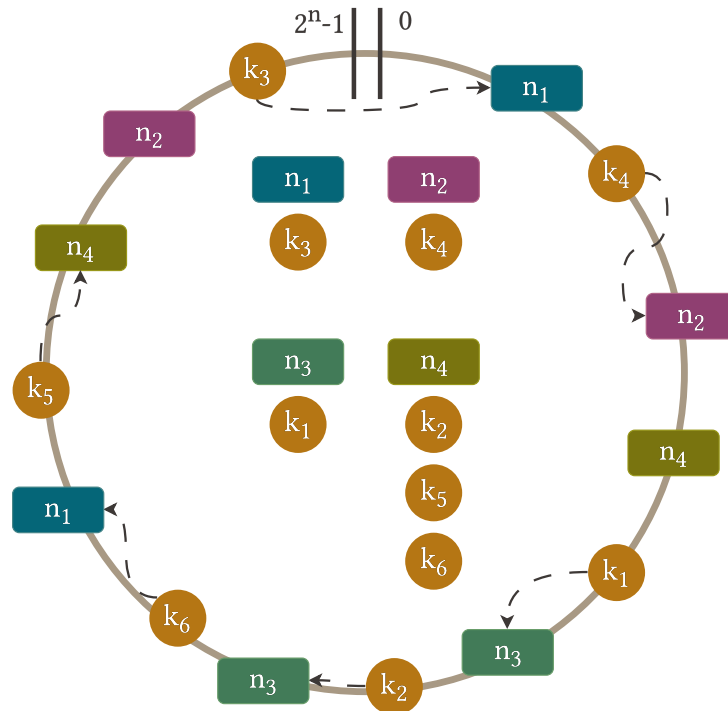


Figure 1: Visualization of consistent hashing as described by Karger et al. [2]

many buckets to ensure uniform key distribution. *Chord*, a distributed lookup protocol developed at the Massachusetts Institute of Technology, introduces the term *virtual nodes* as a synonym for buckets belonging to a node. [4] To achieve a uniform distribution, Chord recommends $O(\log|N|)$ virtual nodes. This, however, has the drawback of requiring more memory for the hash ring data structure, as significantly more virtual nodes than physical nodes need to be stored [5]. Additionally, due to the random distribution of keys, semantic ordering is not preserved, which severely reduces scan efficiency for ordered keys [5]. However, using virtual nodes also provides advantages. When a node becomes unavailable, virtual nodes ensure that orphaned keys are distributed over many nodes instead of only one successor. Conversely, when a node joins the system, it takes ownership of keys from many nodes instead of only one. Lastly, in a system with heterogeneous nodes, different nodes can have different numbers of virtual nodes to reflect their capacity [6].

2.1.2 Highest Random Weight

Around the same time as Karger's work, Thaler and Ravishankar published their consistent hashing algorithm, *HRW*, first as a tech report in 1996 [7] and later as a journal

article in 1998 [8]. Similar to Karger’s consistent hashing, *HRW* was motivated by web proxy caches. Unlike Karger, *HRW* assumes that all clients share the same view of the node set N . It uses a weight function that assigns a weight to each key-node pair, as shown in (3).

$$w_{i,j} = h(K_i, N_j) \quad (3)$$

A key is assigned to the node with the highest weight, *Table I* illustrates this. Compared to Karger’s consistent hashing, *HRW* needs far less memory, as no virtual nodes need to be stored. However, to determine a key assignment, a hash must be calculated for each node of N , while Karger’s consistent hashing requires only a single hash, regardless of the number of nodes in the system [8]. This creates a tradeoff compared to Karger’s consistent hashing: key-to-node lookups have higher compute time at $O(|N|)$, but memory consumption is lower because no virtual nodes need to be stored.

Wang and Ravishankar show that organizing nodes into a tree of virtual clusters and applying *HRW* at each level reduces lookup time to $O(\log|N|)$ at the cost of communicating and organizing additional structure [9].

Thaler and Ravishankar prove the following properties for *HRW* [8]. Let L_n denote the share of keys assigned to node n .

TABLE I

EXAMPLE ASSIGNMENT USING *HRW*. THE HIGHEST WEIGHT PER ROW DETERMINES THE ASSIGNED NODE.

	N_1	N_2	N_3	N_4	Assignment
K_1	28419	87234	41562	19087	N_2
K_2	91445	55102	12089	74821	N_1
K_3	22914	48773	95201	61308	N_3
K_4	38255	71086	29410	83721	N_4

Balance The variance of the per-node load vanishes as the number of keys grows, that is $\text{Var}(L_n) = O\left(\frac{1}{|K|}\right)$. With high probability, each node's load therefore approaches $\frac{1}{|N|}$ when $|K|$ is large.

Minimal Disruption *HRW* achieves the optimal *minimal disruption* bound: on *membership churn*, only a $\frac{1}{|N|}$ share of keys is reassigned, and randomization ensures these keys are spread uniformly across the remaining nodes.

No Replication Since all clients share a consistent view of N and the weight function is deterministic, every key is mapped to exactly one node, ensuring keys are not replicated to other nodes.

HRW can be extended to heterogeneous systems where nodes have different capacities by introducing weights. The weight factor of each node is normalized against the sum of all weights, so that W_j expresses the share of total capacity contributed by node j . The simple approach, used by *Cache Array Routing Protocol (CARP)* [10], is to multiply the hash value by this per-node weight W_j , as shown in (4). While simple, this formulation does not preserve *minimal disruption*: any change to a single W_j shifts the normalization for all other nodes and may cause global remapping [10].

$$w_{i,j} = W_j \cdot h(K_i, N_j) \quad (4)$$

A weighted variant that preserves *minimal disruption* was proposed by Resch [11] with the closed-form scoring function shown in (5). Instead of a normalized share, it operates on the absolute capacity W'_j of each node, so weights do not need to be recomputed when a single capacity changes. $H_{\max}$ denotes the largest value the hash function h can produce (e.g. $2^{32} - 1$ for a 32-bit hash). In this formulation, a change of W'_j only affects the score of node j , so *minimal disruption* is preserved.

$$w_{i,j} = -\frac{W'_j}{\log\left(\frac{h(K_i, N_j)}{H_{\max}}\right)} \quad (5)$$

2.1.3 DxHash

DxHash is a recent consistent hashing algorithm proposed by Dong et al. in 2024 [12]. Motivated by the constrained memory and frequent state changes of *Internet of Things (IoT)* devices, DxHash aims to combine a small memory footprint, high lookup

throughput, and minimal update overhead in a single algorithm. These properties are also useful in other use cases that aim to reduce memory and *CPU* overhead.

DxHash maintains a single boolean array, called *NSArray*, of length m , where m is the smallest power of two greater than or equal to the number of active nodes $|N|$. Each entry in the *NSArray* corresponds to a node ID and is either active (in N) or inactive (reserved for future insertions).

Mapping a key to a node is performed by repeatedly drawing pseudo-random node IDs using a *Pseudo-Random Number Generator (PRNG)* seeded by the key. The first drawn ID that corresponds to an active entry in the *NSArray* is returned as the assignment. Since the *PRNG* output is uniform across the *NSArray*, the probability of hitting an active node in any single round is $\frac{|N|}{m}$, and the expected number of rounds, called the *search length*, is $\frac{m}{|N|}$ [12]. To bound unlucky sequences, DxHash terminates the search after $8m$ rounds, affecting only $\approx 0.03\%$ of keys in practice [12].

Dong et al. prove the following properties for DxHash [12]:

Balance At every iteration of the lookup, the *PRNG* output is uniformly distributed across the active nodes. Each active node therefore has equal probability of being selected.

Minimal Disruption When an active node is removed only the keys that were mapped to it are reassigned, and when a node is added only the keys whose previous winner appeared after the new node in their *PRNG* sequence are remapped. No key ever moves between two unchanged nodes (Theorem 1).

Membership changes are handled by flipping the corresponding *NSArray* entry, which is an $O(1)$ operation. Finding an inactive ID for a new node is more involved. The simple approach reuses the lookup procedure to draw a random inactive entry in expected $O\left(\frac{m}{m-|N|}\right)$ time. A stateful variant maintains an auxiliary queue, called *IQueue*, of inactive IDs for $O(1)$ insertions at the cost of additional memory and the loss of statelessness, since insertion order becomes dependent on removal order [12]. When the *NSArray* fills up, DxHash supports a *scale* operation that doubles its size, and a *shrink* operation that halves it. Both operations remap a non-trivial fraction of keys

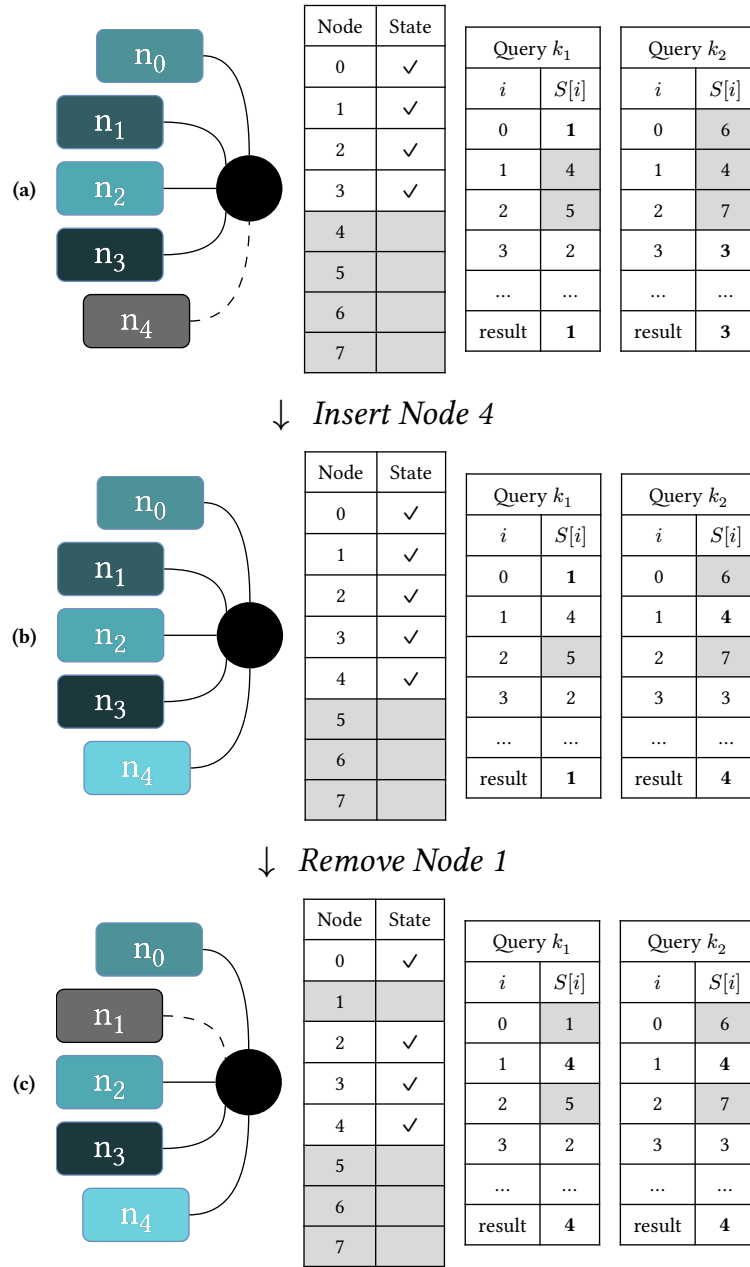


Figure 2: Three lookup states of DxHash: (a) initial four-node cluster, (b) after inserting node 4, (c) after removing node 1. [12]

proportional to the size change. Because scaling the NSArray has high overhead in large systems, Dong et al. recommend scaling in batches [12].

Compared to the other algorithms discussed so far, DxHash combines the strengths of both: like *HRW*, it avoids the virtual nodes of Karger’s consistent hashing and only needs $m/8$ bytes for a bit-packed NSArray [12]. Unlike *HRW*, however, the lookup

does not iterate over all nodes but terminates after an expected $\frac{m}{|N|}$ rounds, which is close to constant when the active ratio $\frac{|N|}{m}$ is high.

Heterogeneous capacities are supported through a weighted variant. Each node j is assigned a floating-point weight $W_j \in [0, 1]$ independently, with inactive nodes having $W_j = 0$. The weights do not need to sum to one. The lookup is extended by a second *PRNG* H producing values in $[0, 1]$. After drawing a candidate node ID j , the lookup additionally draws $H[i]$ and only accepts the node if $W_j \geq H[i]$, otherwise, it continues with the next round. The expected fraction of keys assigned to node j is therefore its weight relative to the total weight of all nodes in the system [12]. Because the acceptance test depends only on the candidate node's own weight, a change to W_j affects only assignments involving node j , preserving *minimal disruption*.

2.2 Distributed Mutual Exclusion

Section 2.1 shows how to achieve *eventually consistent* agreement on assigning a key set K to a node set N . During *membership churn* or node-weight changes, two nodes may simultaneously believe they are responsible for the same key until the changed node set is communicated to all nodes. Depending on the application, this can have mild to severe consequences, from sending an email notification twice to administering the wrong dose of a drug to a patient [13].

To prevent this, mutual exclusion (or, in distributed systems, *distributed mutual exclusion*) must be established. *Kubernetes* already provides the underlying primitive: a totally ordered log via *etcd* (Section 2.3.2), which enables the state-machine-replication foundation discussed at the end of this chapter. The building blocks still needed on top, e.g., *fencing tokens* and *generational clocks*, are introduced first and picked up again by the mutual exclusion mechanism proposed in Section 5.

2.2.1 Fencing tokens

A simple solution to mutual exclusion is the introduction of a central locking service. This service is responsible for handing out locks and informing processes about active locks. However, this does not ensure complete safety. A process may believe it holds a

lock while the central locking service has already released it due to timeouts or other events. This problem can be solved using *fencing tokens*, as shown in Figure 3.

1. When *Alice* reads the object *Foo* (1), the database returns the object including its *fencing token* (2).
2. While *Alice* is processing, *Bob* also reads object *Foo* (3) and receives the same object and *fencing token* in the response (4).
3. Both *Alice* and *Bob* now hold the same *fencing token*, but *Bob* updates *Foo* first (5), triggering an increment of the *fencing token*, which is returned in the database response (6).
4. *Alice* now also wants to update *Foo* (7). Because the *fencing token* has changed in the meantime, this request is rejected (8).

While the term *fencing token* was introduced by a blog post by Kleppmann in 2016 [13], the idea originates from Chubby a decade earlier [14]. Chubby is an internal lock service developed and used by Google. It introduced the concept of a *sequencer*: an opaque byte string that includes a monotonically increasing *lock generation number*. This *sequencer* is included in every request to services. The recipient then checks the *sequencer* against its local Chubby cache or the most recent *sequencer* it has previously observed. The *sequencer* is the direct ancestor of the *fencing token* described above. Equivalent constructions reappear as the revision count of *etcd* resources [15], [16].

2.2.2 Generational clock

Joshi and Fowler introduce the term *generational clock* in their book “Patterns of Distributed Systems” [17] to describe a *logical clock* concept used by many systems.

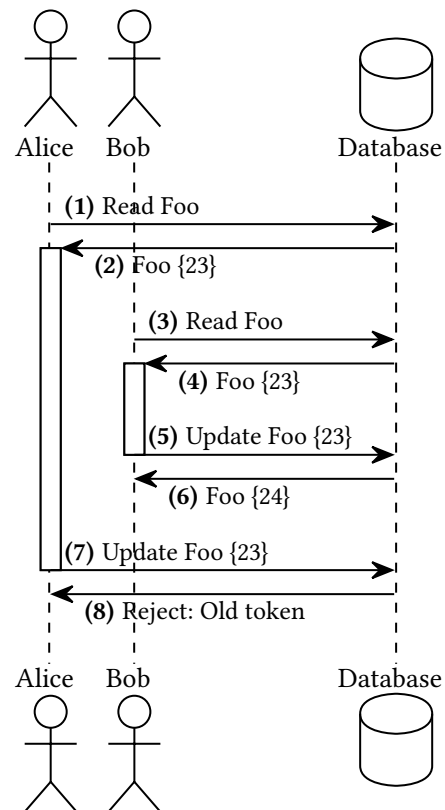


Figure 3: Sequence diagram showing the mechanism of a monotonically increasing *fencing token*.

A *generational clock* is used in leader-follower setups to mark the most recent leader generation. It is an example of a *Lamport Clock* [18], i.e., a monotonically increasing clock that increments on events, where the events are leader elections. After a leader election, every node in a system stores the newly incremented generation number. If the leader then crashes or disconnects from the system, a new leader is elected and the generation counter is incremented again. Should the old leader try to join the system again, its leader heartbeats are rejected because the included generation number is lower than the generation numbers stored in the other nodes.

Many systems implement this concept. For example, in the Raft consensus algorithm¹, a *term* specifies the leader generation [19]. ZooKeeper², a software for distributed coordination, introduces a *zxid* value for each transaction that encodes an epoch counter [20].

2.2.3 State machine replication

The *fencing tokens* and *generational clocks* introduced above are building blocks of a more general pattern: modeling a service as a deterministic state machine and replicating it across nodes. Lamport introduces this idea alongside his logical clock construction (introduced earlier as the Lamport clock) [18]. Given a deterministic transition function $\delta : S \times I \rightarrow S$, every replica that starts in the same initial state s_0 and applies the same sequence of inputs $i_1, i_2, \dots$ traverses the same sequence of states. Achieving distributed mutual exclusion is thereby reduced to a single sub-problem: agreeing on the total order of inputs. As long as all nodes agree on the ordering of inputs, the execution of δ can proceed without nodes interfering with each other [21].

Agreement on input order is provided by consensus protocols such as Multi-Paxos [22] or Raft [19], both of which expose an append-only, totally ordered log to the upper layer. In *Kubernetes*, this role is filled by *etcd*'s revision, surfaced through the *resource version* discussed in Section 2.3.2.

¹<https://raft.github.io/>

²<https://zookeeper.apache.org/>

2.3 Kubernetes

Kubernetes is closely intertwined with the modern shift to cloud-native development and is the leading container orchestration software [23]. The development of *Kubernetes* started internally at Google in the late 2000s, building on lessons from previous internal container orchestration systems, Borg and Omega [24]. After the open-source release of *Kubernetes* in 2014, the *Cloud Native Computing Foundation (CNCF)* was founded and *Kubernetes* became the foundation's first accepted project [25], [26]. Since then, *Kubernetes* has gained significant popularity, with 82% of container users running *Kubernetes* in production [27], and has become a multi-billion-dollar market [28]. This chapter explains the *Kubernetes* architecture and design aspects relevant to this thesis, starting with a high-level overview of the resource model and ending with a detailed breakdown of the underlying communication with *etcd*.

2.3.1 Resource Model

Instead of simply providing an *API* that allows users to *imperatively* execute actions to run and manage containers, *Kubernetes* uses a *declarative* approach through its resource model. The *KRM* is a generalized pattern for building declarative *APIs*, going far beyond the scope of managing containers [29]. The following sections explain the structure of object schemas in *Kubernetes* and the concepts of *operators* and *controllers*.

Similar to the concepts of *classes* and *objects* in object-oriented programming, *kinds* in the *KRM* describe schemas used to define *resource* instances.

Each *kind* in *Kubernetes* is identified by a unique *Group Version Kind (GVK)*. For example, the kind *Pod* has the group and version `core/v1`.

Group The group is the category to which a kind belongs.

Version The version of a kind, which enables multiple versions of a resource kind to exist in a *cluster* without interfering with each other.

Kind The specific kind inside a group.

Additionally, some *API* terminology is important in the context of *Kubernetes* and the *KRM* [30].

Resource type The name used in *Uniform Resource Locators (URLs)*, lowercase and plural by convention. While exceptions exist in the core *Kubernetes API*, in the context of this thesis it can be assumed that there is a one-to-one mapping between *kinds* and *resource types*, e.g. the kind *Pod* has a corresponding resource type *Pods*.

Resource A single instance of a resource type.

Collection A list of resources or instances of a resource type.

Sub-resource Resource types may include sub-resources. For example, most resource types have a *status* sub-resource.

Every *kind* in *Kubernetes* follows a similar schema, as shown in *Code 1*. The fields *apiVersion* and *kind* form the *GVK*. *metadata* contains the name and namespace, used to identify a unique resource in combination with its *GVK*, as well as labels and annotations. Both labels and annotations are untyped key-value pairs whose semantics are determined by developers. The difference between labels and annotations is that labels are used to filter objects, while annotations may contain relevant metadata for *controllers* and cluster administrators.

The schema of the fields described so far is the same for every resource in *Kubernetes*, regardless of kind. Kinds differ, however, in the schema of the field *spec*. This field lists kind-specific information. The example shown in *Code 1* describes a deployment of multiple identical pods. The field *spec.replicas* defines that three pods should be started, and the field *spec.template* follows the schema of kind *Pod*. The result of this deployment is three pods following the spec defined in *spec.template.spec*.

When creating this deployment through the *Kubernetes API*, nothing happens beyond storing the created resources. This is because the *API Server* handling the request only knows the syntax of the created resource; the semantic meaning is provided by software responsible for *reconciling* a *kind*, called a *controller*.

For a *controller*, a resource describes a *desired state*. Its job is to observe the *current state*, compare it to the *desired state*, and perform actions to get closer to it. This is done in a so-called *control loop*. For example, when creating the deployment shown in *Code 1*, the responsible *controller* observes that no corresponding pod is running in the current state and therefore creates three pods according to the defined template. If one

```
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: nginx-deployment
5    namespace: default
6    labels:
7      app: nginx
8    annotations:
9      key: value
10 spec:
11   replicas: 3
12   selector:
13     matchLabels:
14       app: nginx
15   template:
16     metadata:
17       labels:
18         app: nginx
19     spec:
20       containers:
21       - name: nginx
22         image: nginx:1.14.2
23         ports:
24         - containerPort: 80
```

Code 1: *Kubernetes resource example showing a resource of kind Deployment, defining a deployment of three pods that each run an nginx container and expose port 80.*

of the started pods crashes after some time, the controller observes this and replaces it with a new pod to reach the desired state again.

Built-in kinds have their controller included in Kubernetes in a component called the *kube-controller-manager*. However, as mentioned at the beginning, the KRM can be used for much more than the built-in container orchestration functionality. For this, Custom Resource Definitions (CRDs) can be created to define new kinds that are not included in core Kubernetes. To create a CRD, a resource of kind `CustomResourceDefinition` must be applied, containing metadata such as the name

and group of the `CR` and how the resource should be displayed in `kubectl`, the *Kubernetes Command Line Interface (CLI)*. Most importantly, it contains an `OpenAPI`³ v3.0 specification defining the schema of the `CR` [31]. This allows developers not only to define the structure and types of the `CR`, but also to enable complex validations through `OpenAPI` validation and *Common Expression Language (CEL)*⁴ expressions [32], [33]. If even more validation logic is needed, admission webhooks allow `CR` developers to validate resources during runtime [34].

Out of the box, *Kubernetes API* extensions support *create, read, update, and delete (CRUD)* operations and other features known from built-in resources, such as authentication, authorization, and watch connections. However, *Kubernetes* does not provide a *controller* for `CRs`; resource reconciliation must be implemented separately by the `CR` developer. Frameworks like *Kubebuilder* (further explained in Section 2.4) can be used to simplify this development process.

By leveraging this extensibility, the *Kubernetes API* can be used for many purposes, such as managing Postgres clusters through CloudNativePG⁵, managing *Kubernetes* external cloud infrastructure with Crossplane⁶, and even managing *Kubernetes* itself using Cluster *API*⁷.

2.3.2 Application Programming Interface

The *KRM* is exposed by the *kube-api-server* component in the control plane. The *API Server* processes internal requests (e.g. from other control plane components and node kubelets) and external requests (e.g. from developers using the *CLI* `kubectl` and pods running on worker *nodes*) [35]. This central role is also shown in Figure 4, which shows the dependency of nearly all components in a cluster on the *API Server*. The component is responsible not only for persisting *KRM* resources in *etcd*⁸, the distributed key-value store used by *Kubernetes*, but also for handling authentication, authorization, and validation (as described in Section 2.3.1) of requests [36], [37].

³<https://www.openapis.org/>

⁴<https://cel.dev/>

⁵<https://cloudnative-pg.io/>

⁶<https://www.crossplane.io/>

⁷<https://cluster-api.sigs.k8s.io/>

⁸<https://etcd.io/>

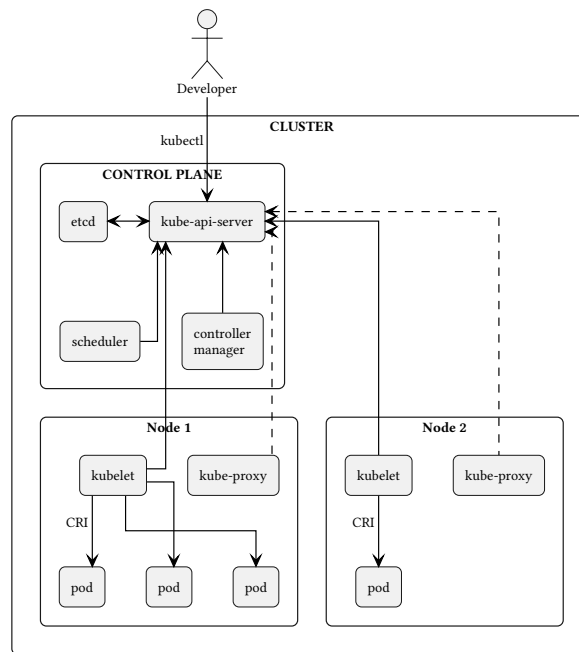


Figure 4: Kubernetes architecture showing the relation between the control plane, Nodes and how a developer interacts with the cluster through the kube-api-server component. [35]

All operations on the *KRM* are expressed as *Representational State Transfer (REST)* requests, with each resource type exposed as an endpoint under `/apis/<group>/<version>/...` (or `/api/<version>/...` for resources in the built-in *core* group) [30].

The *API* supports all *CRUD* verbs shown in Table II, together with a long-running *watch* stream. By passing the query parameter `?watch=true` on a collection endpoint, a *watch* stream is opened and kept alive for 30 minutes or longer [39]. During a *watch* connection, the client receives updates to a collection (creation, update, and deletion of resources) without polling, providing an efficient way to detect changes. This is especially useful for *controllers*, which need to be informed about changes to trigger reconciliations. An example is shown in Code 2: each event contains a type (`ADDED`, `MODIFIED`, `DELETED`, etc.) and the changed object.

The example includes the *resourceVersion* located in the *metadata* property of all *Kubernetes* resources. This parameter is a monotonically increasing counter used as a *fencing token*, similar to the definition in Section 2.2. The *API* Server uses this *resource version* to reject modifying requests that include an old *fencing token*, ensuring clients

TABLE II
VERBS SUPPORTED BY THE *KUBERNETES API* [30], [38]

Verb	Path	Description
GET	<code>/<type>/<name>/<type></code>	Retrieve a resource or collection
GET	<code>/<type>/?watch=true</code>	Watch changes of a collection
POST	<code>/<type></code>	Create a new resource
PUT	<code>/<type>/<name></code>	Replace a resource
PATCH	<code>/<type>/<name></code>	Selectively modify a resource
DELETE	<code>/<type>/<name>/<type></code>	Delete a resource or collection

do not modify resources based on an outdated local view [30]. The *resource version* can also be used to continue a stopped watch request from the last observed event by providing the `?resourceVersion=<resourceVersion>` parameter.

```

1 {
2   "type": "ADDED",
3   "object": {"metadata": {"name": "abc", "resourceVersion":
4     "10596", ...}, ...}
5 }
6 {
7   "type": "MODIFIED",
8   "object": {"metadata": {"name": "abc", "resourceVersion":
9     "11020", ...}, ...}
10 }
```

Code 2: Kubernetes watch request example.

2.3.3 API Server and etcd

As shown in *Figure 4*, the *API* Server is responsible for exposing the *Kubernetes API* and the *KRM* and is accessed by both internal and external clients. For persisting data, *etcd*⁸ is used, which provides a distributed but strongly consistent key-value store. The distributed nature of *etcd* enables horizontal scaling of the control plane, spreading load over multiple *API* Server instances and providing *high availability (HA)* [35].

An *etcd* cluster consists of a single leader, which accepts modifying requests, and followers, which only replicate the leader. The Raft consensus algorithm¹ is used by the leader to have a majority of followers accept write requests before committing. With this design, *etcd* guarantees *Atomicity, Consistency, Isolation and Durability (ACID)* for its key-value *API*, which is crucial for its use as storage in *Kubernetes* [15], [40].

etcd is a key-value store rather than a fully relational database and provides a basic *API* separated into three feature categories:

Key-Value *API* The primary functionality, providing functions to modify keys and key ranges.

Watch *API* Functionality for event-based monitoring of key ranges.

Lease *API* Mechanisms for client liveness checking.

The watch *API* is used by the *API* Server to serve watch requests as described in the previous section. Similar to the *API* exposed by the *API* Server, watch streams in *etcd* are long-running requests that stream events to the client [15]. *etcd* guarantees that watch stream events are *ordered* by their revision, never appear twice (*unique*), and are not dropped (*reliable*). However, *linearizability* is not guaranteed, which means that multiple watch streams can be at different revisions at any time [40].

To reduce load on the *etcd* cluster, the *API* Server does not open a separate watch stream to *etcd* for each *API* Server watch request. Instead, the *API* Server opens a single watch stream to *etcd* per resource type. This design is shown in *Figure 5*. The last n watch events are cached in a *watchCache*. This allows clients to continue or start a watch stream from a specified *resource version* if it is still cached. From the watch cache, each client watch request is managed by a *cacheWatcher*, which is responsible for applying filters specified by the client to the event stream. For example, filters on

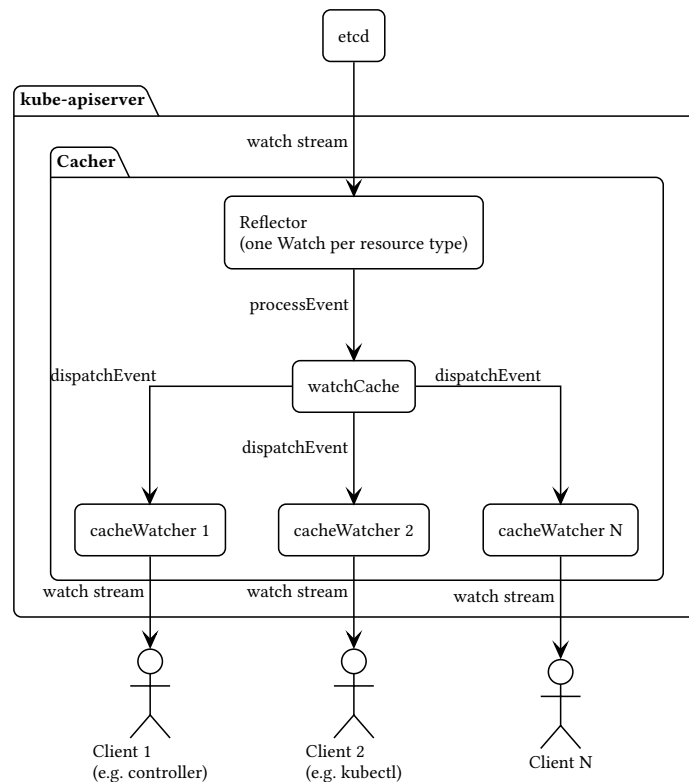


Figure 5: API Server watch request fan out.

namespaces or labels are applied only at this last step; the watch cache and the *etcd* watch stream always listen to all events of a resource type, without filters. A filtered watch request therefore does not reduce load on the *API* Server or *etcd*, but only the bandwidth between the *API* Server and the client [41]. The fan-out architecture greatly improves the efficiency of watch requests. They incur no additional cost after the initial burst of events when creating the stream [42]. Write requests, however, incur additional cost per active watch stream on the modified resource type, as they trigger an event that must be distributed to all watch streams [42].

In general, *Kubernetes* defines cluster limits for *API* Server requests and other cluster limitations worth noting [43], [44]:

1. By default, the *API* Server can handle up to 400 non-mutating (reading resources and collections) and 200 mutating (creating, updating, deleting resources) requests simultaneously. The initial setup of watch requests counts towards the non-mutating quota.

2. Resources may not be larger than 1.5 MB.
3. There may be up to 150,000 resources per resource type for built-in resource types. Other resources may have much lower limits.
4. There may be up to 1.5 GB of resources per resource type.
5. The total size of all resources must be less than 2 GB by default. The maximum *etcd* memory should not be increased above 8 GB.

These limits are the basis for the *Service Level Objectives (SLOs)* that define guarantees for *API* latencies and pod startup times. For clusters exceeding the limits defined above, *SLOs* cannot be guaranteed.

2.4 Kubebuilder

Controllers are the central component in *Kubernetes* responsible for reconciling resources, as explained in *Section 2.3.1*. To implement controllers for *CRs* in Go⁹, developers can use the *Kubebuilder* framework¹⁰, which is developed by the API Machinery Special Interest Group [45]. The same special interest group also develops *controller-runtime*¹¹, the library containing the actual runtime managing *operators*.

This section explains all components relevant to this thesis in detail, including the manager, controller, and their components. *Kubebuilder* also supports implementing webhooks, which can validate modifying requests for *CRs*, as described in *Section 2.3.1*. As this thesis does not use the webhook feature of *Kubebuilder*, its functionality and architecture are not elaborated further.

2.4.1 Architecture

Before elaborating on the individual components, an overview of the *Kubebuilder* architecture, as shown in *Figure 6*, is beneficial. An operator in *Kubebuilder* consists of one *manager*, which handles one or more *controllers*. The manager is also responsible for shared resources such as the *API* Server client and the cache.

⁹<https://go.dev/>

¹⁰<https://kubebuilder.io/>

¹¹<https://github.com/kubernetes-sigs/controller-runtime>

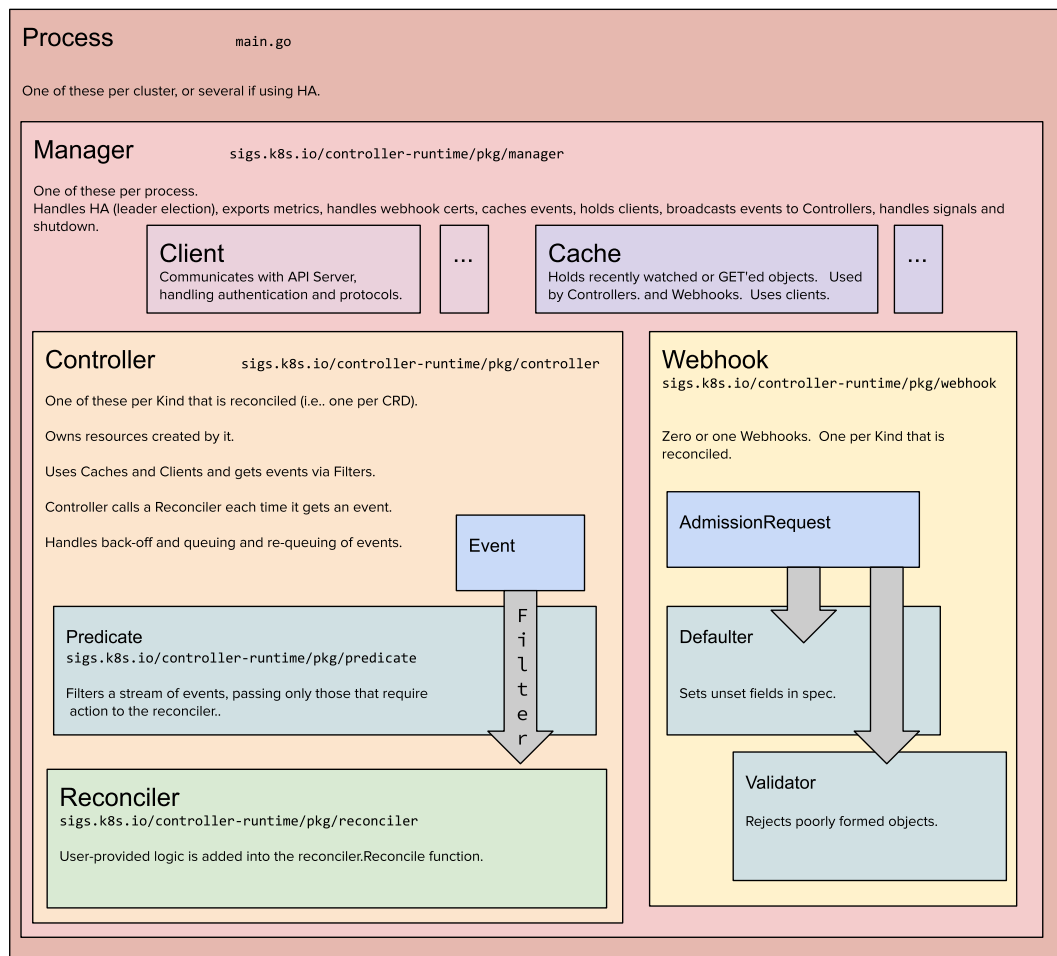


Figure 6: Diagram showing the Kubebuilder architecture [1].

The cache fulfills the read and *watch* interface of an *API* Server client and is passed to *controllers* instead of the actual client for those requests. This means that, by default, *controllers* always read from the cache while writing directly to the *API* Server. To ensure *controllers* do not read stale data, the cache opens *watch* requests for read resources, as described in more detail in *Section 2.4.4*.

Controllers are configured by the developer to *watch* different resources that should trigger a reconcile. Watched resources always include the resource type to be reconciled, but can also include owned resources (e.g. resources that are created from *CR* need to trigger a reconcile on their owner when they change) or other developer-defined relations. *Section 2.4.5* elaborates on this.

Finally, events that should trigger a reconcile are passed to the developer-provided *reconciler*, which contains the actual business logic for reconciling a *CR*.

2.4.2 Manager

The manager is a singleton in the *operator* that manages *controllers* and shared components. It is instantiated from a `NewManager(*rest.Config, manager.Options)` function that receives a *REST* config used to create the client and an options struct that allows configuring all manager components.

In addition to the components shown in *Figure 6*, the manager also opens endpoints for liveness, readiness, and metrics, and manages leader election if enabled.

In essence, the manager's job is to receive runnables added during initialization by the manager itself or by external components (e.g. the *controller* adds itself to the manager) and start them.

The manager exposes a unified `Add(manager.Runnable)` function for this, which can be used to register all sorts of components, including *controllers*, caches, and leader election runnables. *Runnable* functions receive a context and may return an error. They should block until either the context is canceled or an error occurs. When calling `Start(context.Context)`, all registered runnables are started. If either the context provided to `Start(...)` is canceled or a runnable returns an error, the context of all runnables is canceled, triggering a global shutdown of the *operator* and its components.

2.4.3 Client

The client is responsible for making authenticated *API* Server requests. Interaction with the client is done through interfaces that cover different aspects of the *Kubernetes API*, as shown in *Figure 7*: the *Client* interface extends the *Reader* and *Writer* interfaces, among others. A *WithWatch* interface adds a *Watch* function to the *Reader* interface to initialize *watch* requests. The *Cache* interface also extends the *Reader* interface to be usable as a normal read-only client, which is discussed in the following section.

The client can be configured from the manager through a `client.Options` struct or completely replaced by providing a custom `client.NewClient` function used by the manager to initialize a new client.

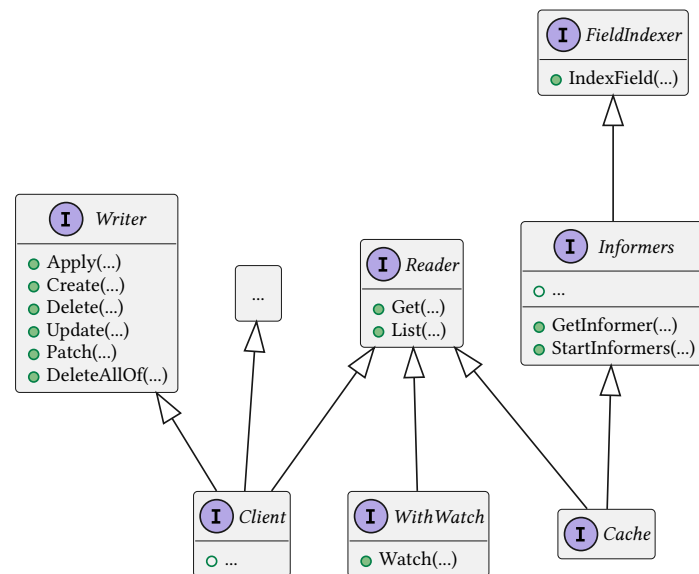


Figure 7: Relationship of client and cache interfaces in the controller-runtime library.

2.4.4 Cache

To reduce read requests to the API Server, Kubebuilder caches accessed resource types. The client exposed by the manager with `GetClient()` is wired to pass *Reader* interface calls to the cache and *Writer* interface calls directly to the client. This allows efficient reads, but there may be situations where reading from the cache is undesired, which can be achieved by using the *Reader* provided by `GetAPIReader()`.

The cache uses two components to provide its functionality: *indexers* and *informers*. Informers handle API Server communication. For a given GVK, they first *sync* their state by making a list request. Once synced, a *watch* request is opened with the retrieved *resource version*. Event handlers can be added to informers to trigger functions on add, update, and delete events.

The second component used by the cache, indexers, are created for each cached GVK and hold the actual resources. They subscribe to informer events to keep the cache up to date and provide functionality to retrieve resources. Additionally, as the name suggests, field indexes on spec fields of resources can be added to this component, allowing quick resource queries.

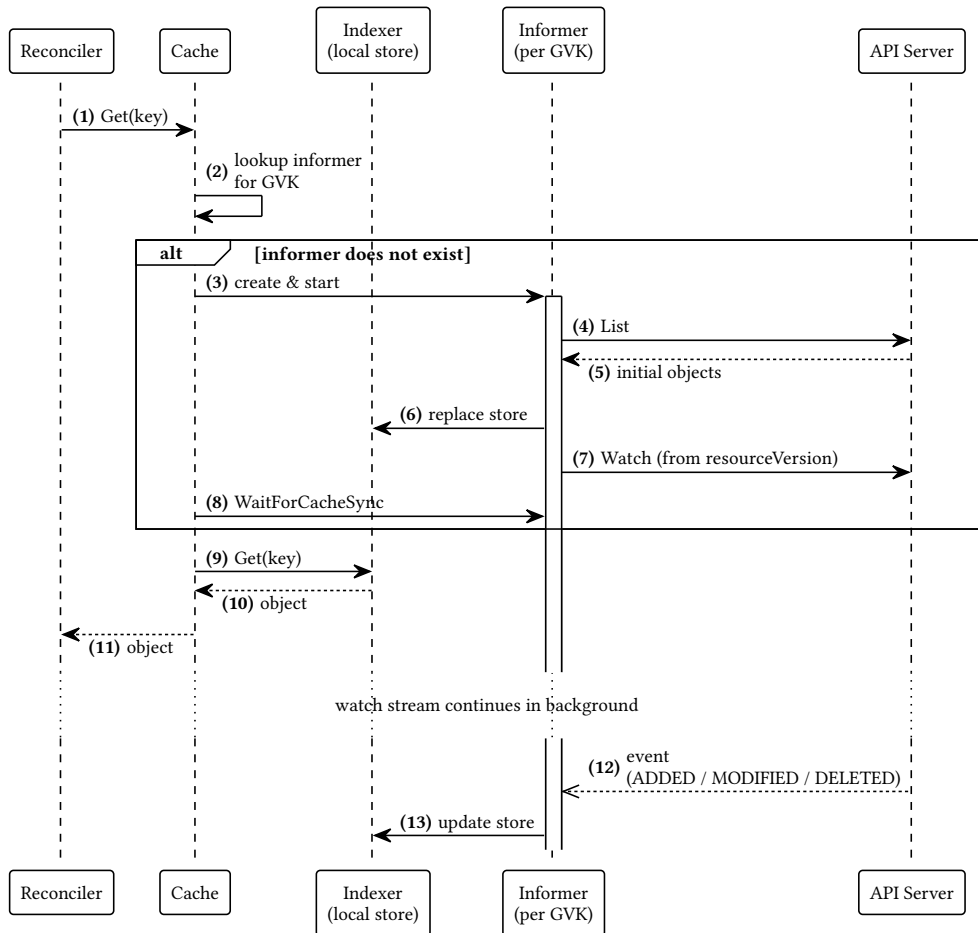


Figure 8: Data flow of a `Get` request against the controller-runtime cache. The informer is started lazily on the first request for a GVK and keeps the local indexer in sync via a watch stream.

Figure 8 illustrates the data flow of a cache-served `Get(key)`. On the first request for a given GVK, the cache lazily instantiates an informer, which performs an initial `List` against the `API` Server, populates the local indexer, and then opens a `Watch` starting from the returned `resource version`. Subsequent events from the `watch` stream are applied to the indexer in the background, so all further `Get` calls are served locally without hitting the `API` Server.

Two manager options can be used to replace the default cache implementation: `NewCache` allows replacing the whole cache implementation, and `NewInformer` can be used to provide a custom `SharedIndexInformer` component builder holding an indexer and informer pair for a given GVK.

2.4.5 Controller

The previous sections described global components, e.g. components managed by the manager and therefore *operator* singletons. An *operator* contains one or more *controllers*, with each *controller* responsible for one resource type.

Controllers implement the *Runnable* interface and are added to the manager's runnables. This allows the manager to control starting and gracefully stopping *controllers*.

As reconciles can be triggered not only by the reconciled resource itself but also by changes to other resources, the following functions configure which events a *controller* should reconcile:

For This function defines the resource type the *operator* should reconcile. Filters can be passed to only reconcile specific changes.

Owns *Kubernetes* resources can have an owner reference referencing their owner resource. With **Owns**, the owner resource is reconciled on changes to owned resources.

Watches This function allows defining custom relationships. An event handler receives change events of the watched resource and returns the reconcile requests that should occur due to the change of the watched resource.

WatchesRawSource A raw source can be used to create reconcile requests that do not originate from *Kubernetes* resource events.

Additionally, predicates can be specified to filter resource events and prevent unnecessary reconciliations. As an example, this can be used to filter events without *resource version* changes, which prevents reconciles on status changes.

Reconcile requests that pass the predicates are added to the *controllers* work queue. The work queue deduplicates requests, so the same resource is never in the queue twice. With the *controller* option **NewQueue**, a custom queue implementation can be passed to the *controller*.

A *controller* can run multiple reconciles concurrently; the maximum concurrency can be specified by the developer. As long as the maximum concurrency is not reached,

reconcile requests are dequeued from the work queue and passed to the reconcile function. For simplicity, this is described as workers, with each worker handling one reconcile request concurrently.

The reconcile function is the heart of the controller. *Code 3* contains a minimal example of a reconcile function. The reconcile request passed to the reconcile function only contains the namespace and name of the resource to reconcile. However, as discussed in the previous section, the cache allows cheap access to resources.

The reconcile function does not receive information about what triggered the reconcile. However, because reconcilers are implemented idempotently, reconcile functions are designed to reconcile the current state of a resource rather than the changes made to it.

After reconciling, the reconciled resource can be enqueued again with an optional delay. Returning an error also requeues the request with incremental backoff.

The `manager` and controller options mentioned above, in particular `NewCache`, `NewClient`, and `NewQueue`, are the extension points the horizontal-scaling library in *Section 5* plugs into.

```
1 func (r *Reconciler) Reconcile(ctx context.Context, req
   ctrl.Request) (ctrl.Result, error) {
2     // Business logic
3     ...
4     return ctrl.Result{
5         // Optional parameter to requeue after a duration.
6         RequeueAfter: 10 * time.Minute,
7     }, nil
8 }
```

Code 3: Example of a reconcile function that requeues the reconciled resource after 10 minutes.

3 Existing Solutions

There are already several related projects and theses on horizontally scaling *Kubernetes operators*. This chapter details different approaches and solutions to this problem, explaining their advantages and shortcomings. Later chapters use and improve upon the solutions presented here.

3.1 Static Sharding

Static *sharding* describes the simplest form of horizontal scaling. For static *sharding*, resources are marked manually with the assigned *shard*, and each *shard* only watches assigned resources.

Redistribution of resources must be done manually when *shards* or resources are added or removed. It is therefore not viable for dynamic scaling and also does not provide *HA*: if a *shard* fails, resources are not automatically reconciled by a replacement.

For static load-balancing purposes, the approach can nevertheless be useful, as shown by FluxCD¹² [46]. FluxCD is continuous delivery software that allows declarative and version-controlled management of *Kubernetes* resources. In FluxCD, each *shard* is assigned a label to watch, and the user is responsible for assigning *shard* labels to resources that should be reconciled.

3.2 KubeVela

KubeVela¹³ is a *CNCF* application delivery platform built on top of *Kubernetes* that lets platform teams define higher-level application abstractions and deliver them across environments through custom *controllers*.

The solution presented by KubeVela removes the need for manual labeling of resources by running *shards* in a leader-follower constellation. The leader is not elected but deployed separately. Similar to the FluxCD implementation, each follower watches a resource label to find assigned resources. Unlike FluxCD, resources do not need to be

¹²<https://fluxcd.io/>

¹³<https://kubvela.io/>

labeled manually, but are labeled by a mutating webhook running as a singleton in the leader [47].

The webhook is called by the API Server before a resource is created and allows the leader to add a label to the created resource. While this reduces manual labor, it does not enable dynamic scaling or HA. When removing a shard, resources are not reassigned and manual changes to the assignment labels are needed. Adding a shard does not trigger redistribution of resources; only newly created resources may be assigned to the new shard. Also, when the leader crashes, no new resources can be created until a replacement is started [47].

3.3 Tim Ebert’s Master Thesis

The master’s thesis of Tim Ebert, titled “Horizontally Scalable Kubernetes Controllers”, follows the same premise as this thesis: There is no solution for horizontally scaling Kubernetes operators that only requires slight modifications to the operators source code [48]. In his work, Ebert proposes an enhanced design of the solution used by KubeVela. Like both previous solutions, Ebert’s shards only watch resources with a specific label, and, as with KubeVela, an external component is responsible for assigning these labels to resources.

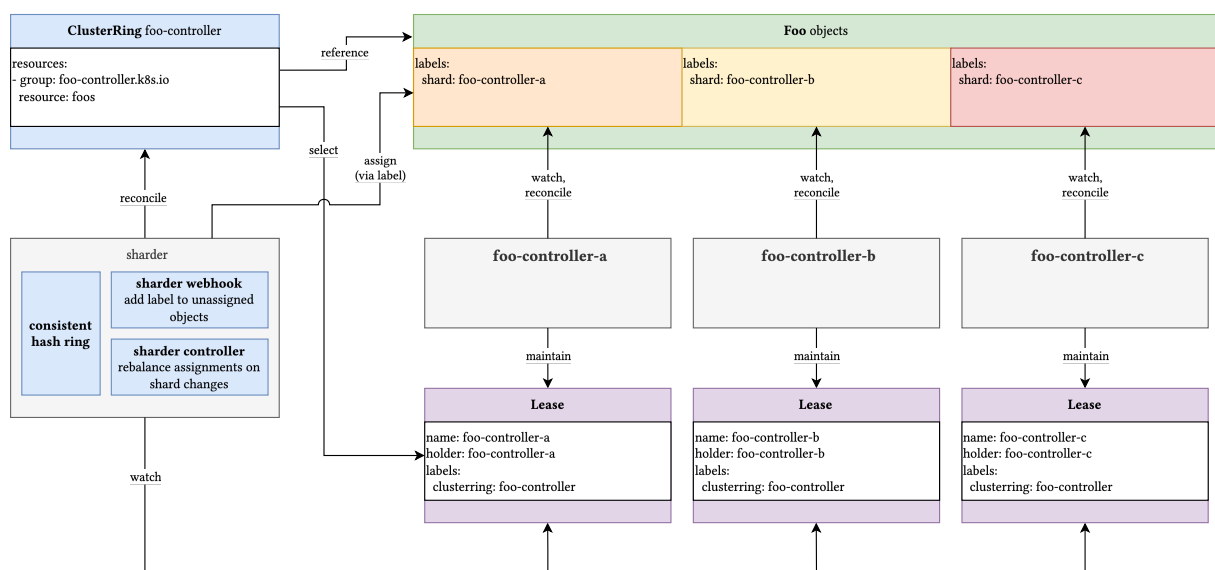


Figure 9: Architecture of the horizontal-scaling solution by Tim Ebert [48].

Figure 9 displays the architecture suggested by Ebert, showing the external *sharder* and the sharded *controllers*. The sharder reconciles a *ClusterRing CR* that contains information about which resource types are reconciled by a ring of *controllers*. Next, the sharder watches leases for a cluster ring, which inform it which and how many *controllers* shards are alive. With this information and a consistent hashing algorithm, each resource can be assigned a *shard* label defining which *shard* is responsible for it. The *controllers* then watch their specific label to only reconcile assigned resources.

Ebert’s solution is the first discussed dynamic and fully autonomous solution. No manual interaction is needed when adding or removing *shards*, and resources are re-distributed automatically on scale-out. To deterministically assign resources to shards, Ebert uses a consistent hash ring with virtual nodes, as described in Section 2.1, which ensures *minimal disruption on membership churn*. The deterministic behavior ensures that no resources need to be reassigned if the sharder is restarted.

To transition resources from one *controller* to another, a two-phase handover is performed: First, a draining label is added by the sharder to all resources that need to vacate a *controller*. Once the *controller* has drained a resource with a draining label (e.g. stopped reconciling it and removed it from its queue), it removes both the draining label and its *shard* label. The sharder is then free to reassign the resources to new *controllers*. This ensures mutual exclusion of *controllers*, so that a resource is never reconciled concurrently [48].

Because each *controller* only watches resources matching its *shard* label, memory consumption is kept minimal for each *shard*. Ebert shows that the combined memory consumption of all *shards* is only slightly higher than the consumption of a singleton instance, while load capacity increases linearly [48].

However, Ebert’s solution needs an additional deployment for the sharder and a *ClusterRing CR* with extra configuration. This adds significant complexity and maintenance overhead, as well as a single point of failure.

3.4 ArgoCD

ArgoCD¹⁴ is another continuous delivery software for declarative *Kubernetes* resource management. As such, ArgoCD faces similar scaling problems, especially when managing not only a single *Kubernetes cluster* but hundreds or more.

To handle reconciling many *clusters*, ArgoCD supports horizontal scaling by distributing target *clusters* over multiple *shard* instances. Previous versions already supported crude horizontal scaling, but with issues such as imbalanced *shards* and heavy rebalancing on membership changes. Since version 2.9.0, *dynamic cluster distribution* is supported as alpha functionality, providing a more stable and balanced approach [49].

Generalized *dynamic cluster distribution* works by communicating the *shard* number via a shared *ConfigMap*. With every *shard* knowing the other *shards*, the assignment from *cluster* to *shard* can be calculated locally. In ArgoCD, a bounded-load consistent hash ring is used as the assignment function, which is a modified Karger consistent hashing algorithm. This algorithm prevents unbalanced distributions of keys by defining an upper relative load each *shard* can handle, with the downside of no longer fulfilling *monotonicity*: On *membership churn*, resources may move between *shards* that are neither joining nor leaving the *fleet*.

With each *shard* only handling a subset of *clusters*, *API* Server connections and cache size are kept small, reducing load on both the *operator shards* and the *clusters API* Servers.

What makes this implementation special is its use of a shared *ConfigMap*, which is a resource type in *Kubernetes* that can be described as an *any* interface: the spec holds generic key-value mappings without typed syntax. The *ConfigMap* used by ArgoCD contains three rows for each *shard*, representing a lease:

ShardNumber An increasing number from 0 to the number of *shards*.

ControllerName The hostname of the *shard*.

HeartbeatTime The last “lease” update.

All *shards* watch this config map and run the following protocol on every readiness probe:

¹⁴<https://argoproj.github.io/cd/>

1. Read the *ConfigMap*
2. Reconcile the row count against the current replica count (fetched from the actual deployment resource):
 - If too few rows, append empty entries.
 - If too many rows, **reset** the whole table to a default (empty) shape so that all pods will re-claim.
3. If this pod already owns a row (matched by `ControllerName == hostname`) refresh its `HeartbeatTime`.
4. Otherwise, take the first row that is either unassigned (`ControllerName == ""`) or whose `HeartbeatTime` is older than `HeartbeatTimeout = 3 * HeartbeatDuration`, write this pod's hostname into it, and update `HeartbeatTime`.
5. Write the *ConfigMap* back.

If a different *shard* writes the *ConfigMap* in the meantime, the protocol is retried. As described in *Section 2.3.2*, the *API* Server ensures mutual exclusion when writing resources.

This approach achieves balanced load distribution; however, the *ConfigMap* “lease” ensures neither *minimal disruption* on down-scaling nor mutual exclusion. When a *shard* is removed, all *shards* need to add themselves to the *ConfigMap* again, shuffling all *cluster* assignments. Even more detrimental, when a *shard* joins, it may add itself to the *ConfigMap* and start reconciling before existing *shards* notice the joined *shard* and stop reconciling. This inadvertently leads to concurrent reconciliation of resources, which is noted by the developers and documented in an issue [50].

3.5 multicluster-runtime

The *multicluster-runtime*¹⁵ is a project of the *multicluster* special interest group of *Kubernetes*. It is a library extending *controller-runtime* with support for reconciling resources in many *clusters* from one *controller*.

¹⁵<https://github.com/kubernetes-sigs/multicluster-runtime>

This can again lead to scaling issues. Therefore, the multicluster runtime implements *sharding* on the *cluster* level, similar to the ArgoCD approach described in the previous section.

Instead of using a single *ConfigMap* for peer discovery, a lease is created for each *shard*. All *shards* watch for leases of peers and emit events when peers join or leave. Leases also have built-in support for heartbeats with a `RenewTime` field in the lease spec that holds the last renewal time.

The local assignment of *clusters* to *shards* is implemented similarly to ArgoCD by using a consistent hashing function. However, the multicluster runtime uses *HRW*. This achieves balanced load while also ensuring *minimal disruption*, as described in *Section 2.1*.

While ArgoCD does not prevent concurrent reconciles during membership transitions, the multicluster runtime implements a solution to this problem. In addition to the leases created for each *shard*, a lease is created for each *cluster*. Each *shard* assigns itself to the cluster leases it is responsible for. On transition events, the *shard* taking over a *cluster* may only start reconciling when the previous *shard* has removed itself from the lease or when the lease expires.

This ensures that even when *clusters* are transitioned from one *shard* to another, they are never reconciled concurrently. However, due to maintaining one lease per *cluster*, *API Server* load increases linearly with the number of *clusters* to distribute.

3.6 Comparison

The surveyed solutions cover a broad spectrum, from purely manual assignment to fully autonomous *sharding* with per-resource mutual exclusion. *Table III* summarizes the five approaches along the properties that matter most for a horizontally scalable *Kubernetes operator*: whether the assignment adapts to *membership churn* without human intervention (*Dynamic*), whether a failing *shard*'s resources are picked up by a surviving *shard* (*HA*), whether the number of reassigned keys on *membership churn* achieves *minimal disruption*, whether the assignment additionally satisfies *monotonicity*, whether the design prevents two *shards* from reconciling the same resource

TABLE III

COMPARISON OF THE SURVEYED SHARDING APPROACHES. A CHECKMARK (✓) DENOTES THAT THE PROPERTY IS FULFILLED, A DASH (—) DENOTES THAT IT IS NOT.

Approach	Dynamic	<u>HA</u>	Minimal Disruption	Monotonicity	Mutual Exclusion	External Component
Static Sharding (FluxCD)	—	—	—	—	—	—
KubeVela	—	—	—	—	—	Webhook
Ebert	✓	✓	✓	✓	✓	Sharder
ArgoCD	✓	✓	✓	—	—	—
multicluster-runtime	✓	✓	✓	✓	✓	—

concurrently (*Mutual Exclusion*), and whether the design requires deploying components in addition to the sharded operator itself (*External Component*).

Static Sharding and KubeVela require manual involvement or a leader-follower topology that does not react to membership churn, so all rebalancing properties are void. Ebert’s design fulfills every dynamic property but at the cost of a dedicated sharder deployment and its accompanying CRs. ArgoCD achieves dynamic redistribution without an external component with minimal disruption on membership churn. However, the bounded-load extension explicitly forfeits monotonicity, and the shared-*ConfigMap* protocol does not enforce mutual exclusion, as evidenced by the documented concurrent-reconciliation issue [50]. Only the multicluster-runtime combines dynamic redistribution, HA, minimal disruption, monotonicity, and mutual exclusion without an external component, but it significantly increases API Server load. The design proposed in *Section 5* targets the same combination of properties for the general CR sharding case.

4 Methodology and Approach

Traditional scientific methodologies present challenges for computer science research, as conventional science emphasizes explanation and understanding of phenomena, whereas computer science focuses on developing practical solutions for real-world problems. The design-science paradigm addresses this challenge by providing a problem-solving-oriented methodology.

The paradigm’s objective, as articulated by Hevner et al. [51], is to “extend the boundaries of human and organizational capabilities by creating new and innovative artifacts.” This thesis adheres to the seven established guidelines of the paradigm as follows:

1. **Design as an Artifact:** This thesis proposes an architectural design for generalized horizontal scaling of *Kubernetes operators* and implements it as an extension for *Kubebuilder*. The *Kubebuilder* implementation is evaluated to prove its usefulness. The artifacts, i.e. the designed architecture and library implementation, therefore aim to solve the identified problem or improve the existing situation.
2. **Problem Relevance:** *Section 1* describes the problems that occur without horizontally scalable *Kubernetes operators*. This, together with the sustained interest in cloud deployment and container orchestration, establishes the relevance of solving the problem. *Section 4.1* identifies the environmental constraints and states the functional (FR) and non-functional (NFR) requirements the solution design must address, following *ISO/IEC/IEEE 29148:2018* [52] and the *Easy Approach to Requirements Syntax (EARS)* notation [53].
3. **Design Evaluation:** The effectiveness, efficiency, and correctness of the created implementation are rigorously evaluated in *Section 6* against the seven evaluation objectives *EO-1* through *EO-7* defined in *Section 4.2*, each stated as a *GQM* triplet [54], together answering *RQ 2*. Using “proof by construction” [55], the designed architecture is transitively evaluated as well.
4. **Research Contributions:** *Section 3* makes the lack of complete solutions apparent. All existing approaches are missing one or more aspects needed to satisfy the functional requirements defined in *Section 4.1*. This thesis provides an architecture

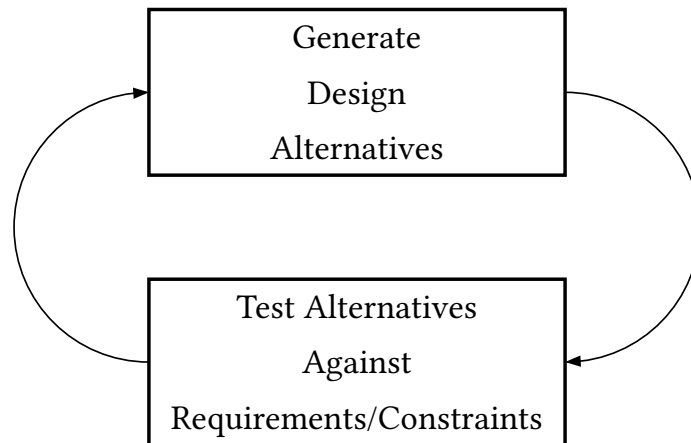


Figure 10: The generate/test cycle used in the search process according to Hevner et al. [51].

and implementation satisfying all functional requirements, thereby answering *RQ 1* and contributing to the existing research field of horizontally scalable *Kubernetes operators*.

5. **Research Rigor:** The design and evaluation of the artifact must be reliably reproducible. Reproducibility is ensured through the detailed artifact description in *Section 5*. The evaluation methodology is reproducible through the requirements and measurement procedures stated in *Section 4.1* and *Section 4.2*, respectively, together with the benchmarking results reported in *Section 6*.
6. **Design as a Search Process:** Artifact design constitutes a search process involving the identification of solutions using available means to achieve desired ends within environmental constraints. Means represent available resources and tools, ends encompass goals and solution constraints, while laws constitute uncontrollable environmental restrictions. *Section 4.1* elaborates on solution and environment requirements and constraints, and *Section 4.2* states the comparative objectives against which candidate designs are judged. The search process is iterative, as illustrated in *Figure 10*. Generated artifacts require testing, with insights informing subsequent improvement cycles. Because several evaluation objectives (memory, *CPU*, request, and reconcile overhead) admit no absolute optimum and are pursued only relative to a single-instance baseline, a *satisficing* [56] approach is adopted without exhaustive enumeration of all possible solutions. Nevertheless, all design decisions are explained and justified to ensure process transparency.

7. **Communication of Research:** Knowledge dissemination from the search process requires effective communication and preservation. This thesis includes a thorough explanation of the fundamentals and systems needed to understand the contributed artifacts in *Section 2*. This allows a wider community of developers to understand and use the findings of this thesis.

4.1 Requirements and Environmental Constraints

To ensure the search process for a solution is as precise and directed as possible, requirements and environmental constraints for the designed architecture and implemented library must be identified. Together, they formalize the design constraints of *RQ 1* as verifiable pass/fail obligations.

The library introduced by this thesis is constrained to *Kubernetes operators* following the architecture of *Kubebuilder* and using *controller-runtime*. While the general concept may be adaptable to other frameworks or runtimes, this thesis does not discuss them to keep the scope manageable. Additionally, *controllers* need to be designed to handle omitted events. For example, during the transition of a resource from an *operator* instance $\mathcal{A}$ to an instance $\mathcal{B}$, multiple updates may occur to the resource, but $\mathcal{B}$ will only receive a single *generic* event. With an idempotent design, which is already required by *Kubebuilder* [1], this is not a problem. However, a second implication is that if the resource is deleted during the transition from $\mathcal{A}$ to $\mathcal{B}$, the deletion will never be reconciled, even after the transition finishes. If an *operator* needs to reconcile on deletion of a resource, finalizers¹⁶ are needed to prevent premature deletion.

Beyond that, no environmental constraints are imposed on this work; the produced library should work with all *operators* bound by the above statements, which should include all *operators* following *Kubebuilder* best practices.

The requirements of the proposed concept and library are stated according to *ISO/IEC/IEEE 29148:2018* and are accordingly split into *functional requirements* (denoted as *FR-n*) and *non-functional requirements* (denoted as *NFR-n*) [52]. Requirements additionally follow *EARS* as described by Mavin et al. [53], resulting in machine-parseable,

¹⁶<https://book.kubebuilder.io/reference/using-finalizers.html>

standardized prose in which each requirement is a single *shall* (or *shall not*) obligation with binary pass/fail verification. Softer, comparative properties that cannot be expressed as a binary threshold are relegated to the evaluation objectives in *Section 4.2*.

4.1.1 Functional Requirements

FR 1 Mutual Exclusion: The library *shall* ensure that no *CR* instance is reconciled concurrently by more than one *shard*, both during steady state and during resource handovers caused by *membership churn*.

Mutual exclusion is the correctness property that makes horizontal scaling safe: without it, two *shards* could issue conflicting writes for the same resource and violate the sequential-reconciler assumption that underpins *Kubebuilders* best-practice reconciler design [1]. This requirement is trivially satisfiable by leader election (all reconciliation performed by a single *shard*), but doing so degrades the load-distribution objective *EO 1* to its worst case.

FR 2 Failsafe on *API Server* Disconnection: If a *shard* loses its connection to the *Kubernetes API Server*, whether temporarily or permanently, then the *shard shall* shut down.

Healthy *shards* take over resources from *shards* that stay disconnected long enough for their lease to expire; without the failsafe, the disconnected *shard* would continue reconciling in parallel and violate *FR 1*.

FR 3 No Additional Network Communication: The library *shall not* open any additional network ports on *operator* pods, and all coordination between *shards shall* occur exclusively through the *Kubernetes API Server*.

This preserves compatibility with existing network policies and pod specifications, and avoids the operational overhead of introducing a separate coordination service into deployment topologies.

4.1.2 Non-Functional Requirements

*NFR 1 **Kubebuilder Compatibility:*** The library *shall* integrate as an opt-in extension of any *operator* that follows *Kubebuilder* best practices, without modifying the *reconciler* contract prescribed by *Kubebuilder*.

Because *Kubebuilder* already requires idempotent reconcilers [1], the omitted-event behavior described in the environmental constraints above is absorbed by the existing contract. Adoption cost, monetary and in terms of time and complexity, is thereby bounded by the code required to enable the library, not by rewriting existing controllers.

4.2 Evaluation Objectives

To test the improvement or overhead of the proposed solution, a simple example *operator* is developed. This *operator* is then tested with and without horizontal scaling.

The example *operator* reconciles the resource type *websites*, which defines a website that should be pinged at a specified interval. *Code 4* shows an example of such a resource. The *operator* reconciling this resource would make a *GET* request to <https://example.com> every 10 seconds. The response code and latency are stored in a status subresource. By using a service like HTTPBin¹⁷, the desired request latency can be specified in `spec.url`, which allows tight control over the duration of a reconcile. Using a deliberately low reconcile interval of 10 seconds, far shorter than the change-driven or 24-hour cycles typical of production operators, allows the benchmark to simulate the load a real-world operator would only encounter with a far greater number of resources, keeping the evaluation practical.

The requirements in *Section 4.1* state binary correctness properties of the library. To evaluate qualities that cannot meaningfully be expressed as a pass/fail threshold, this thesis follows the *GQM* approach [54]. Each objective is stated as a triplet of the *goal* pursued, the *question* the measurement answers, and the *metric* used to answer it,

¹⁷<https://httpbin.org>

```

1 apiVersion: monitoring.marvin-fuchs.de/v1alpha1
2 kind: Website
3 spec:
4   url: https://example.com
5   interval: 10s

```

Code 4: Example of a Website resource, retrieving <https://example.com> every 10 seconds.

together with a measurement procedure. The objectives collectively make RQ 2 measurable. Objectives are numbered *EO-n* and referenced from Section 6. All objectives are measured relative to the number of resources reconciled and the number of *shards* in the *fleet*, so that scaling behavior can be observed.

As *Kubernetes* workloads generally run on Linux, all measurement procedures assume a Linux host. Time-based metrics use a monotonic clock, as wall clocks can be skewed and stepped by *Network Time Protocol (NTP)* or manual input [57]. Go's `time.Now()` returns both a wall and a monotonic reading [58], and comparing two monotonic readings yields accurate intervals irrespective of wall-clock adjustments [57]. Container-level resource metrics are read from the Linux kernel's cgroup v2 interface [59], exposed via cAdvisor¹⁸ unless higher accuracy than cAdvisor's one-second sampling is required, in which case the corresponding cgroup files are read directly.

EO 1 Load Fairness:

Goal Characterize the fairness of reconcile-load distribution across *shards* in a horizontally scaled *operator*. The minimum *node* size an *operator* needs is bound by the *shard* using the most resources. With fair load distribution, the overall maximum resource usage is reduced, allowing *operators* to run on smaller and cheaper *nodes*.

Question How evenly is reconcile time distributed across *shards* for different numbers of shards and resources?

Metric Each *worker i* on a *shard* exposes a monotonically increasing counter $c_i(t)$ that accumulates the total time it has spent reconciling resources up to time

¹⁸<https://github.com/google/cadvisor>

t . Over an observation window $[t_0, t_0 + \Delta t]$, a shards reconcile utilization U is defined in (6) as the fraction of available worker-time spent actively reconciling resources:

$$U = \frac{\sum_{i=1}^k (c_i(t_0 + \Delta t) - c_i(t_0))}{k \cdot \Delta t} \quad (6)$$

where k is the number of workers on the shard and $k \cdot \Delta t$ is the total worker-time available in the window, yielding $U \in [0, 1]$. Fairness across a fleet of n shards with utilizations $U_1, \dots, U_n$ is quantified by Jain's fairness index [60], shown in (7), which yields a value in $(0, 1]$:

$$J(U_1, U_2, \dots, U_n) = \frac{(\sum_{i=1}^n U_i)^2}{n \cdot \sum_{i=1}^n U_i^2} \quad (7)$$

A value of 1 indicates a completely fair distribution ($U_1 = \dots = U_n$); a value of $\frac{1}{n}$ indicates all load concentrated on a single shard. Leader-election deployments are pinned to $J = \frac{1}{n}$ by construction and provide the lower-bound baseline against which the library is compared.

EO 2 Memory Overhead:

Goal Quantify the additional memory an operator pod consumes when the horizontal-scaling library is enabled, so that operator authors can weigh the memory and thereby monetary cost of adoption.

Question How does the Resident Set Size (RSS) of a shard-enabled operator pod compare to that of a single-instance baseline with respect to the number of resources reconciled and the number of shards in the fleet?

Metric Memory is reported as RSS, the anonymous memory (stack, heap) resident in physical RAM for the operator container [61, p. 58]. RSS is chosen over the broader Working Set Size (WSS) because file-backed pages the kernel keeps cached for eviction are not memory the library itself allocates and are subject to unrelated kernel behavior; excluding them isolates the memory attributable to

the *operator* process [62]. Container-level *RSS* is exposed by cgroup v2 via the *anon* field of *memory.stat* [59] and by cAdvisor as *container_memory_rss* [62].

EO 3 *CPU Overhead*:

Goal Quantify the additional *CPU* time consumed and thereby additional cost produced by an *operator* pod attributable to *kube-sharder*.

Question How does the accumulated *CPU* time of a shard-enabled *operator* pod compare to that of a single-instance baseline, as a function of the number of resources reconciled and the number of *shards*?

Metric *CPU* time is the time a process was scheduled on a *CPU* during a given interval. cgroup v2's *cpu.stat* interface exposes user, system, and throttled time in microseconds [59]. cAdvisor also exports these counters but only at one-second granularity; the measurement procedure therefore reads *cpu.stat* directly from the cgroup filesystem. Because throttling would introduce workload-independent noise, no *CPU* limits are configured on *operator* pods and the throttled-time counter is verified to remain zero throughout each benchmark run.

EO 4 *API Server Request Overhead*:

Goal Quantify the additional load the library imposes on the *Kubernetes API* Server. As defined in Section 2.3.3, the *API* Server has *SLOs* that must not be exceeded. Any additional *API* Server overhead therefore decreases the remaining load available to other users.

Question How many additional *API* Server requests does the library issue per unit time and per resource reconciled, split by *read*, *watch*, and *write* operations?

Metric Requests are intercepted at the *Kubernetes* client by installing a custom *WrapTransport* around the *operators* HTTP round-tripper. Every outgoing request and its response is observed, categorized into *read*, *watch*, and *write*, and attributed to either the user's *reconciler* or the library itself based on the calling

code path. This yields exact request counts, distinguished by origin, without relying on server-side metrics.

EO 5 **Storage Overhead:**

Goal Quantify the additional storage consumed in the *Kubernetes* cluster's backing store as a result of the library's coordination metadata. As described in *Section 2.3.3*, *etcd* has upper storage limits, so storage overhead decreases the *etcd* storage available for other usage.

Question How much additional data does the library persist in *etcd* (or an alternative backing store) per *shard*, per *CR*, and per *fleet*?

Metric The actual on-disk size in *etcd* is implementation-dependent; *Kubernetes* stores objects in Protobuf, achieving roughly a 2x reduction relative to the equivalent *JavaScript Object Notation (JSON)/Yet Another Markup Language (YAML)* representation [63]. A conservative upper bound on storage overhead is therefore obtained by summing the *YAML* serialization of every additional object or field introduced by the library.

EO 6 **Reconcile Latency Overhead:**

Goal Quantify the additional latency introduced between the arrival of an event at the *operator* and the invocation of the user's *reconciler*.

Question By how much does the library increase the delay from event ingress to reconcile start, compared to a single-instance *operator*?

Metric An event traverses several stages before the *reconciler* runs, as detailed in *Section 2.4*; the library inserts additional stages that may extend this pipeline. A monotonic timestamp is captured at event ingress and at reconcile entry using `time.Now()`, and the difference is recorded per event. Distributions (median and tail percentiles) are compared between the shard-enabled and single-instance configurations.

EO 7 Failover Time:

Goal Quantify the improvement in failover time delivered by horizontal scaling relative to a single-instance *operator*, which is the primary functional benefit the library targets. Reduced failover time can reduce downtime defined by *Service Level Indicators (SLIs)*, making it possible to achieve tighter *SLOs* and thereby save costs and increase revenue.

Question How long does a *fleet* take to become stable again after one or more *shards* fail, and how does this compare to the recovery time of an unsharded, leader-elected *operator*?

Metric A *fleet* is considered *stable* once all resources that required reconciliation as a consequence of the *shard* failures have been reconciled and only new events remain to be handled. Failover time is the interval, measured on a monotonic clock, from an injected *shard* failure to fleet stability. Because absolute failover time depends on the per-reconcile duration, the number of concurrent *workers*, and the number of resources in the *fleet*, comparisons are performed with identical *reconcilers* and identical per-reconcile work.

5 Concept

This chapter details the components and functionality of the *kube-sharder* library and thereby answers *RQ 1*. First, the architecture is described to provide an overview of important components and their relationships. Next, core components are explained in more detail, with a particular focus on the final section describing the revision control mechanism that ensures the core requirement of mutual exclusion at all times.

5.1 Architecture

The architecture of *kube-sharder* is designed to provide drop-in replacements for components of *controller-runtime* that developers use to construct an *operator*, fulfilling

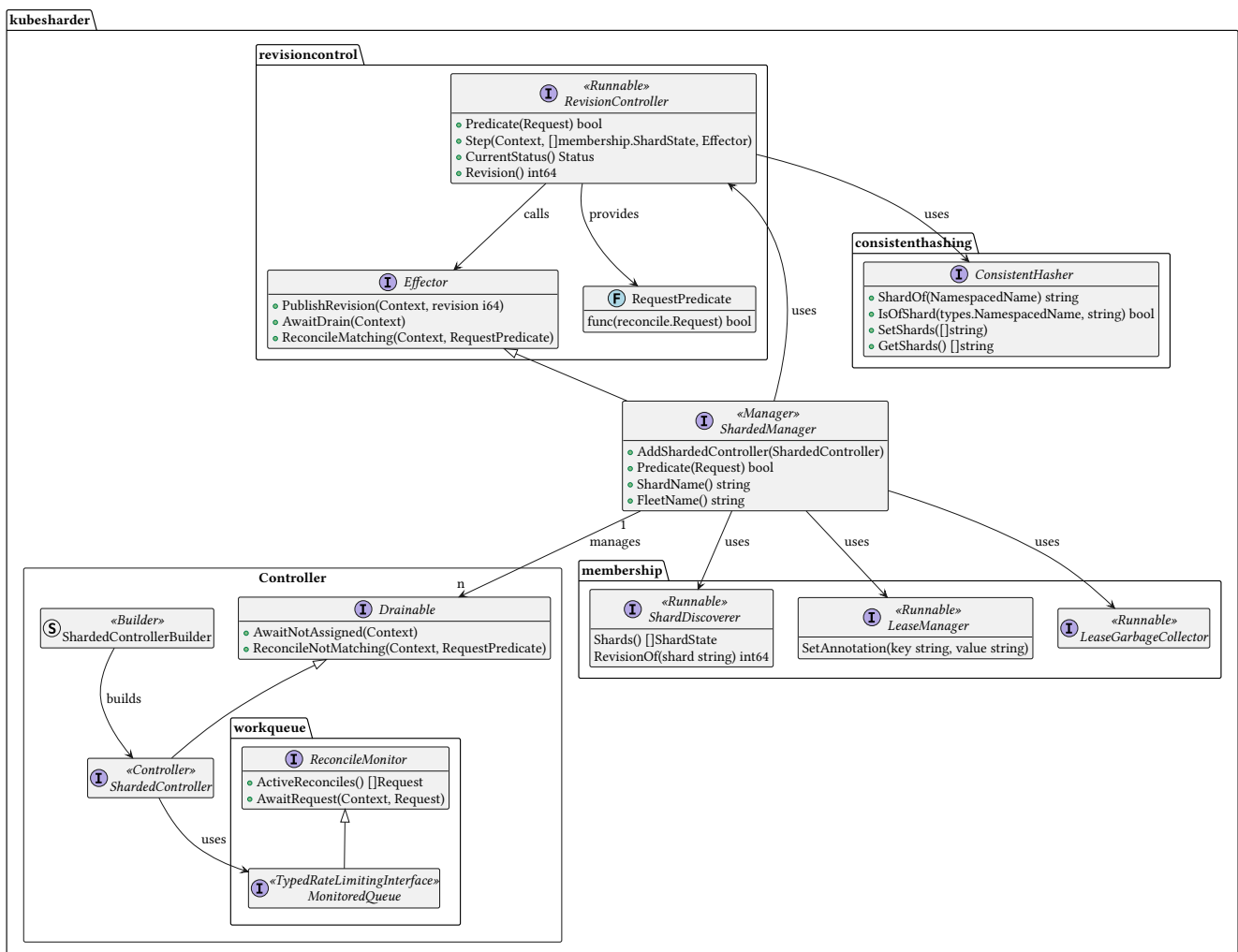


Figure 11: Class diagram showing the architecture of the *kube-sharder* library.

NFR 1. This includes the interfaces `ShardedManager` and `ShardedController`, which replace `Manager` and `Controller`, as well as `ShardedControllerBuilder`, which replaces `ControllerBuilder`, the factory abstraction used to produce *controllers*.

The architecture of *Kubebuilder operators* can be separated into two layers, as described in *Section 2.4*. The manager level is responsible for shared components such as the *API* Server client and cache. In the sharded configuration, the manager is additionally responsible for handling membership-related tasks (described in more detail in *Section 5.2*), providing a consistent hashing implementation through the revision control components, and managing sharded informers if enabled, as described in *Section 5.4*.

The package `membership` contains three components responsible for managing leases and discovering peers. *Section 5.2* describes these components more thoroughly. In `revisioncontrol`, events caused by *membership churn* are processed, and the component exposes a request predicate that other components use to determine whether a *shard* is responsible for a resource. All these packages and components live on the manager level and are therefore singletons in the *operator*. The manager is responsible for wiring these components together and managing their lifecycle. Components managed by the manager implement the `Runnable` interface, which exposes a `Run` function that blocks until an error occurs or the manager shuts down.

Components of the `membership` package fulfill *FR 3* by communicating *fleet* information solely through the *API* Server rather than directly with peers.

The interface `ConsistentHasher` provides a unified interface for interacting with different consistent hashing implementations. By default, *kube-sharder* uses an implementation of *HRW* as its consistent hashing algorithm. While *HRW* has a higher compute overhead than Karger consistent hashing when many *shards* are involved, it provides even balancing of resources without requiring many virtual nodes, as described in *Section 2.1*. DxHash would yield similar load balancing while requiring less compute overhead, but it requires *shards* to be numbered absolutely. This would only be possible if an *operator* were deployed as a `StatefulSet` or with another external ordering mechanism, making DxHash unsuitable for a library that must integrate into any *Kubebuilder operator* with minimal changes.

The *controller* level contains components that exist in every started *controller*. The `ShardedController` itself is a runnable and is thereby started and managed by the manager. It additionally implements the `Drainable` interface needed for the revision control logic described in *Section 5.5*. The actual *operator* sharding occurs on the *controller* level in the `MonitoredWorkqueue`, which wraps the default *controller-runtime* work queue and filters out reconcile requests not assigned to the shard. This is explained in more detail in *Section 5.3*.

To create a sharded *controller*, the `ShardedControllerBuilder` is used, which closely follows the *API* of the `ControllerBuilder` provided by the *controller-runtime*.

5.2 Membership Discovery

To reliably calculate the assignment of resources to *shards*, each *shard* needs to know its peers. *Kubernetes* provides a resource type, *leases*, that can be used for this problem, similar to the implementations of Ebert (*Section 3.3*) and the multicluster runtime (*Section 3.5*). Lease objects hold information about their time to live and a last refresh timestamp. If the refresh timestamp is older than the time to live, the lease is considered *expired*. *kube-sharder* uses three cooperating components to publish, reclaim, and observe these leases.

All three components implement `manager.Runnable`, which allows the components to trigger an *operator* shutdown in case of *API* Server connection loss. This fail-fast contract is important because, as described in *FR 2*, a *shard* that has lost its *API* Server connection must stop reconciliation so that its peers, once the lease expires, can safely take over its assigned resources.

Lease Manager Creates and maintains the *shard*'s own lease. Besides static options to configure the *shard* and *fleet* name as well as lease durations, the component allows annotations to be set dynamically; this is used in *Section 5.5* to communicate the revision version to peers. After being started, the lease manager creates its lease and enters the refresh loop; both initial creation failure and refresh failure surface as errors from `Run` and thereby trigger the fail-fast shutdown described above.

Lease Garbage Collector Deletes expired leases. On startup, it opens an informer on all leases of the configured *fleet*. When a lease expires, the collector issues a delete request that races with peers detecting the same expiration; the first *shard* succeeds while the others receive a *Not Found* response, which is treated as success locally. This keeps the *API* Server free of stale leases and lets the *shard* discoverer detect leaving members by listening for deletion events rather than tracking expiration timestamps itself.

***shard* Discoverer** Provides upstream services access to the current *fleet* state and exposes an update channel used to notify consumers of *membership churn*. On start it opens an informer to listen for changes to *fleet* leases. On watch events it compares the previously observed *fleet* with the newly observed set and triggers an update if they diverge.

5.3 Monitored Workqueue

Running in every *controller*, the work queue provides a buffer for reconcile requests. The *controller-runtime* work queue is deduplicated and allows for scheduled and prioritized requests. Requests are added to the work queue by informers listening to *API* events and retrieved by concurrent controller workers.

The monitored work queue wraps the default *controller-runtime* work queue to intercept calls to its add and retrieval functions. The request predicate exposed by the revision controller is applied to filter requests both entering and exiting the queue. While filtering on retrieval alone would suffice for correctness, filtering on add also prevents the work queue from being filled with requests not assigned to the *shard*.

The monitored queue also plays a vital role during *membership churn* events. *Figure 12* shows how the sharded controller interacts with the monitored queue to safely drain resources no longer assigned to the shard. First, the predicate is updated to reflect the new *fleet*; second, the manager waits for all controllers to stop reconciling resources no longer assigned to the shard.

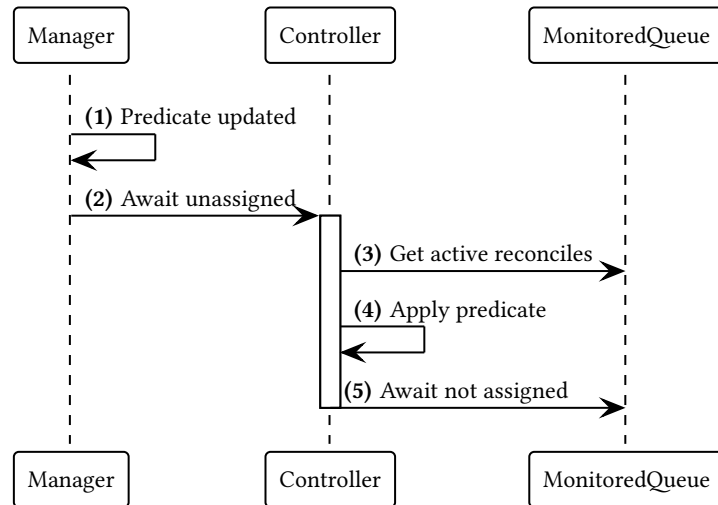


Figure 12: Sequence diagram showing how the controller interacts with the wrapped work queue.

To achieve this, each controller first retrieves the list of resources currently being reconciled and filters this list using the updated predicate. Any resource no longer assigned to the shard is awaited, blocking until its reconciliation finishes.

With this design, the monitored work queue combines two core functionalities in a lightweight component: ensuring unassigned resources are not reconciled, and allowing safe resource transitions by awaiting in-flight reconciles. It thereby forms one of the two parts needed to fulfill *FR 1*, achieving mutual exclusion for stable fleets.

5.4 Sharded Cache

As described in *Section 2.4*, all controllers share a client-side cache that is automatically created for requested resources. The cache implementation provided by controller-runtime caches all resources of a GVK, including those not assigned to the shard. This causes high memory overhead, as each shard holds a cache containing the resources of every shard, scaling overall memory usage linearly with the number of shards.

To reduce this memory overhead, which is one of the evaluation objectives as described in *EO 2*, a sharded cache is needed. This sharded cache must implement two additional functionalities:

1. Determine, for any resource of any GVK, whether it should be stored in the cache.

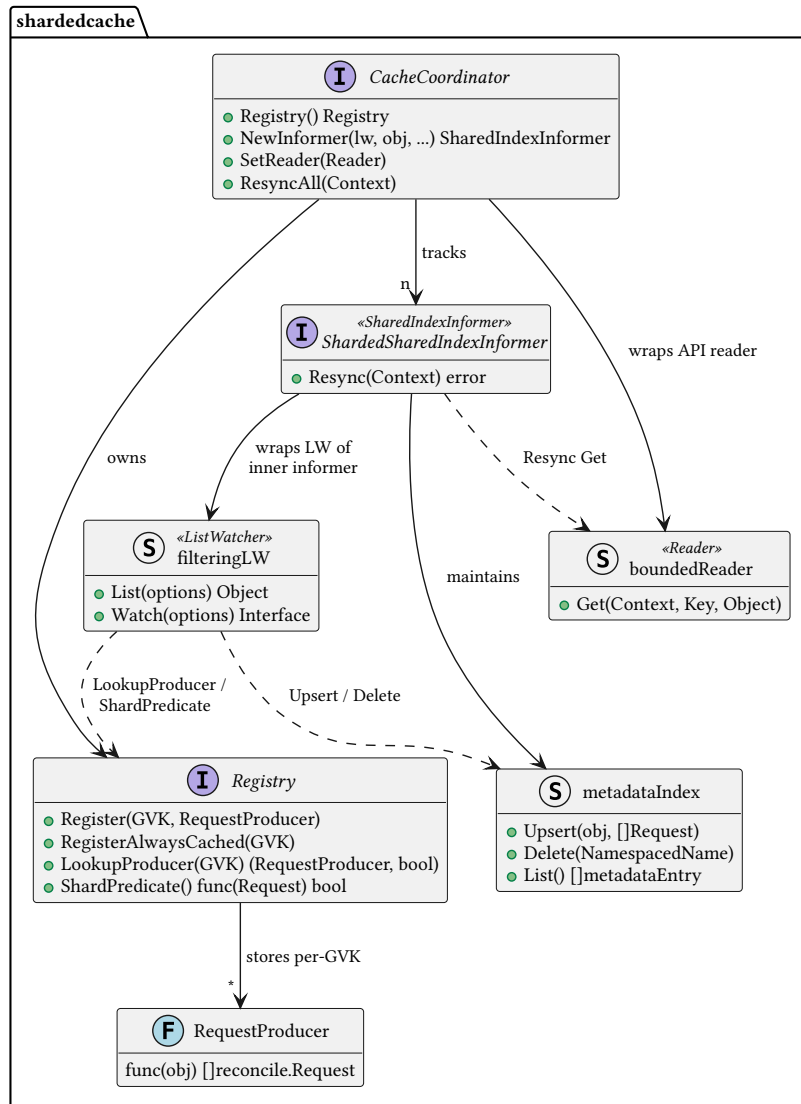


Figure 13: Class diagram showing the architecture of the sharded cache.

2. Resync the cache on *membership churn*, removing resources no longer assigned and adding newly assigned ones.

In the following, these two functionalities are explained using the class diagram shown in Figure 13.

5.4.1 Sharding

The `CacheCoordinator` is owned by the manager, and the exposed `Registry` is the component used before the manager starts to register sharded and unsharded re-

sources. When using the controller builder to create a controller, three functions can be used to specify which resources the controller should watch:

For The given resource type should be watched, and on changes, reconcile requests for the resource should be created. This also specifies the resource type the controller reconciles.

Owns The given resource type should be watched, and on changes, reconcile requests for the **owning** resource should be created. The type of the owning resource is the reconciled type of the controller (specified with *For*).

Watches The given resource type should be watched, and on changes, a custom *mapping* function is called that converts the watched resource to one or more reconcile requests for the reconciled type of the controller (specified with *For*).

All three functions have in common that they produce one or more *reconcile requests* from watch events. This can be used to determine whether a resource should be stored in the cache. The sharded controller builder registers the watched resources using the `Register` function, providing the watched resource *GVK* and a request producer matching the function that was called.

A resource must be held in the cache if any of the reconcile requests created from the registered `RequestProducers` match the manager's request predicate. To make this more concrete, the following examples show resources that should and should not be held in the cache:

1. For a controller *for* pods, pod resources are held in the cache if they match the request predicate.
2. A controller *for* config maps *owns* secrets. A secret is held in the cache if its owning config map matches the request predicate.
3. A controller *for* deployments *watches* config maps. A config map is held in the cache if any reconcile request created from the *mapping* function matches the request predicate.

If two controllers register `RequestProducers` for the same *GVK*, only one of the produced reconcile requests must match the request predicate. This ensures that all controllers have all needed resources available.

GVKs that are not registered are not sharded at all: because no sharding decision can be made without additional information from the developer, it must be assumed that an unregistered resource must be accessed by all shards. An edge case is one where one controller registers a GVK that another controller needs unsharded access to. For example, one controller may reconcile secrets while another controller requires access to one specific secret, regardless of the shard. Without any changes, this shard-shared secret would be available only to a single shard. To prevent this, a `RegisterAlwaysCached` function is provided. Using this function, GVKs excluded from sharding can be registered explicitly.

Using the information stored in the `Registry`, calls to `NewInformer` on the `CacheCoordinator` decide whether the informer for the requested GVK needs to be sharded. If not sharded, a default `controller-runtime SharedIndexInformer` is created; otherwise, a `ShardedIndexInformer` is returned. This sharded informer wraps the provided `ListWatcher` with a sharding `filteringLW`. On list and watch events, it runs the checks described above using the `Registry`, and depending on whether the event is assigned to the shard, either passes it on to all subscribers or only to the `metadataIndex`, which is explained in the following section.

This design reduces memory consumption significantly without breaking existing implementations by reusing information that is already provided through the builder functions described above.

5.4.2 Resyncing

Sharding alone does not ensure a correct cache during membership churn. After the request predicate changes, resources may need to be deleted from the cache or re-fetched from the API Server. To achieve this, the `CacheCoordinator` holds a reference to every `ShardedSharedIndexInformer` it has created and exposes a `ResyncAll` function that delegates to the `Resync` function of each index informer.

Each index informer contains not only an index holding the cached resources, but also a metadata index holding the namespaced name and resulting reconcile requests of *all* resources, both cached and discarded. This information is much smaller than complete objects.

TABLE IV

ACTION MATRIX DESCRIBING THE REQUIRED ACTION FOR A RESOURCE DEPENDING ON WHETHER IT IS CURRENTLY CACHED AND WHETHER IT MATCHES THE PREDICATE DURING CACHE RESYNC.

Predicate \ Cache	match	no match
stored	nothing	delete
not stored	fetch	nothing

Using this index and the updated request predicate, the required action for each resource can be determined, as shown in *Table IV*. To delete a resource from the cache, a synthetic delete event is dispatched. To re-add a resource, it is fetched from the *API Server* and a synthetic create event is dispatched.

While this reduces the memory overhead, it increases the load on the *API Server* during *membership churn*, as every transferred resource is re-fetched from the *API Server*. This results in approximately $\frac{r}{s}$ requests for r managed resources and s shards, as described in *Section 2.1*.

5.5 Revision Control

This section describes *revision control*, the mechanism used by *kube-sharder* to ensure mutual exclusion between *shards* during *membership churn* without per-resource *API Server* writes. It is the second part needed to fulfill *FR 1*.

5.5.1 Problem

One core requirement of *kube-sharder* is to ensure no concurrent reconciles of the same resource. In a stable *fleet*, where every peer agrees on the observed *shards*, this can be achieved simply by having a disjoint function for the resource-to-*shard* assignments, as described in *Section 5.3*.

Because the *watch* requests used by the *shard* discoverer of each *shard* are only eventually consistent, *shards* may have different views of the *fleet* during *membership*

churn. If not fenced by some mechanism during transition moments, multiple *shards* may believe they are responsible for the same resource.

5.5.2 Design Space

To ensure mutual exclusion during *membership churn* and safe transitions of resources between *shards*, some kind of *fencing token* is required. The solutions described in *Section 3* already provide some ideas on how to solve this problem:

- Ebert uses labels on resources to communicate *shard* ownership. A new *shard* can only start reconciling a resource once the old *shard* has removed its label.
- The multicluster runtime manages a lease per resource. A new *shard* can only start reconciling a resource once the lease is released or expired.

The downside of both solutions is increased *API* Server load. In Ebert's solution, transitioning a resource between *shards* requires two write requests to the *API* Server: one to remove the old label and a second to add the new label. The multicluster runtime, in addition to making two write requests per resource transition, also makes periodic write requests to maintain the resource leases.

While negligible for a single resource, the number of write requests to the *API* Server scales as $O(n)$ in the resource count. Since *API* Server overhead is one of the metrics used to evaluate the presented solution, a different approach is needed.

5.5.3 Revision Version

Inspired by the fencing token used for *Kubernetes* resources described in *Section 2.3.2*, the *resource version*, *kube-sharder* introduces a monotonically increasing *generational clock*: the *revision version*. This revision version is maintained locally by each *shard* and incremented every time a *shard* observes a change in the *fleet*. By observing the published revision versions of peers, each *shard* determines which peers have accepted the new *fleet* and are ready to transition resources between *shards*.

Modeling the revision control mechanism as a state machine is a concrete use of Lamport's *state machine replication* pattern: if the total order of inputs is the same for a replicated state machine, all replicas converge on the same state, which can be used to achieve mutual exclusion. By observing the leases maintained by each *shard* and

communicating the revision version through those leases, the *Kubernetes API* guarantees that the watch requests of each *shard* receive change events in a total order, as described in *Section 2.3.2*.

5.5.4 State Machine

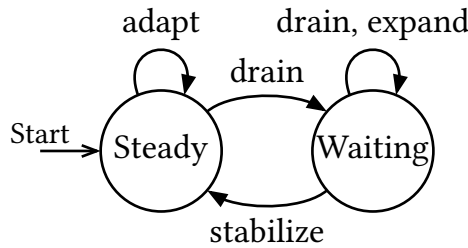


Figure 14: *Revision control state machine.*

The *revision control* component, which manages the revision version and the resource-assignment function, is modeled as the state machine shown in *Figure 14*. It consists of two states, *Steady* and *Waiting*, and four transitions: *drain*, *expand*, *stabilize*, and *adapt*.

On startup the state machine begins in state *Steady* with the last seen *fleet* being empty; in this state a *shard* reconciles all resources assigned to it according to its local view of the *fleet*. *shards* transition to and stay in the *Waiting* state on changes to the *fleet* until all peers acknowledge the new set. The transitions are described formally and in detail in the following.

5.5.5 Notation

The following symbols are used to describe the transition functions.

s	The current shard.	transitions are enabled per the state machine.
S	The <i>shard</i> space; the set of all shards.	
$\mathbb{N}$	The set of natural numbers.	$U_s \subseteq S$ Set of <i>shards</i> currently observed by <i>shard</i> s .
R	The resource space; the set of all resources.	$L_s \subseteq S$ Last observed <i>fleet</i> of <i>shard</i> s ; used to detect membership changes.
$\mathbb{B}$	The set $\{\top, \perp\}$ of boolean values.	$L_s^{\text{old}} \subseteq S$ Snapshot of L_s taken at the start of the most recent drain; used
$\sigma_s \in \{\text{Steady}, \text{Waiting}\}$	Control state of <i>shard</i> s ; determines which	

to determine which peers previously held resources that s intends to acquire.	$h : R \times S \times \mathcal{P}(S) \rightarrow \mathbb{B}$ Sharding predicate. $h(r, m, U_m) = \top$ if resource r is assigned to <u>shard</u> m under observed fleet U_m .
$a_s \in \mathbb{N}$ Revision counter of <u>shard</u> s ; incremented on every drain.	$p_s : R \rightarrow \mathbb{B}$ Local ownership predicate of <u>shard</u> s ; $p_s(r) = \top$ if s currently claims r .
$C_s = \{m \in U_s \setminus \{s\} : a_m \geq a_s\}$ Peers caught up to s 's revision counter and thereby ready to hand off resources to s .	$q_s : R \rightarrow \mathbb{B}$, $q_s(r) = \exists m \in \Delta_s : h(r, m, L_s^{\text{old}})$ Predicate identifying resources previously held by peers in Δ_s under the pre-drain membership.
$H_s \subseteq S$ Peers already handled by a previous expand of s within the current drain cycle; reset on every drain.	
$\Delta_s = C_s \setminus H_s$ Peers newly ready since the last expand; the peers processed by the next expand.	

5.5.6 Transitions

Each transition is specified in the precondition/effect style of Lynch's I/O automata [64]: an action fires when its **precondition** holds, applying the **effect** as a single atomic step. Following Lynch and Tuttle's original notation, assignment is written $\leftarrow$; conjuncts of a precondition are separated by semicolons; right-hand sides refer to pre-state values; and state components not mentioned in the effect are unchanged.

The transitions are presented in the order of the normal *drain cycle* (*Drain*, *Expand*, *Stabilize*); *Adapt* is a recovery transition used only when a shard's local view is already consistent but its revision counter lags behind peers due to interrupted watch requests.

Drain

Draining unassigned resources: the ownership predicate is changed to match resources assigned both under the previous *and* the newly observed fleet. In other words, the predicate is narrowed to match only resources that are neither transitioning away from nor to the shard. The revision version is then incremented and published to

communicate to peers that the *shard* has released ownership of unassigned resources and is ready to take ownership of new resources.

$$\begin{aligned}
 \text{Precondition: } & U_s \neq L_s \\
 \text{Effect: } & L_s^{\text{old}} \leftarrow L_s \\
 & L_s \leftarrow U_s \\
 & p_s \leftarrow h(\cdot, s, U_s) \wedge p_s \\
 & a_s \leftarrow a_s + 1 \\
 & H_s \leftarrow \emptyset \\
 & \sigma_s \leftarrow \text{Waiting}
 \end{aligned} \tag{8}$$

Expand

Partial expansion: claim resources previously held by peers that became ready *since the last expand* within the current drain cycle. Peers already accounted for by an earlier expand (tracked in H_s) are excluded, so repeated expands never re-process the same peer. The final precondition conjunct requires that at least one peer still lags behind, so *Expand* is disabled once *Stabilize* can complete the cycle.

$$\begin{aligned}
 \text{Precondition: } & \sigma_s = \text{Waiting}; U_s = L_s; \Delta_s \neq \emptyset; \exists m \in U_s \setminus \{s\} : a_m < a_s \\
 \text{Effect: } & p_s \leftarrow p_s \vee (h(\cdot, s, U_s) \wedge q_s) \\
 & H_s \leftarrow H_s \cup \Delta_s
 \end{aligned} \tag{9}$$

Stabilize

Full expansion: once every peer has caught up, s 's ownership predicate collapses to the current sharding function, returning to state *Steady*.

$$\begin{aligned}
 \text{Precondition: } & \sigma_s = \text{Waiting}; U_s = L_s; \forall m \in U_s \setminus \{s\} : a_m \geq a_s \\
 \text{Effect: } & p_s \leftarrow h(\cdot, s, U_s) \\
 & \sigma_s \leftarrow \text{Steady}
 \end{aligned} \tag{10}$$

Adapt

Adapting the revision version: increase the revision version to the largest revision version observed among peers, provided the fleet did not change. This edge case occurs on missed events after watch reconnects, or during the initialization of a shard: the shard's local view is already consistent with its last observed set, but its revision counter lags behind peers that have already moved on. Because $U_s = L_s$, the shard is already in a steady state and can safely catch up without a drain.

$$\begin{aligned} \text{Precondition: } & \sigma_s = \text{Steady}; U_s = L_s; \exists m \in U_s \setminus \{s\} : a_m > a_s \\ \text{Effect: } & a_s \leftarrow \max_{m \in U_s \setminus \{s\}} a_m \end{aligned} \quad (11)$$

5.5.7 Correctness

Using the defined transition actions, the following shows that mutual exclusion holds at all times and that a stable fleet (no membership churn) eventually leads back to state *Steady*.

Mutual Exclusion

Mutual exclusion must hold for any combination of states as well as during transition actions at all times. To show correctness of the proposed revision concept, each state and transition is discussed from the point of view of a shard (*the shard*) and its peers across different states.

In a stable fleet, where all shards agree on the observed view, mutual exclusion reduces to the sharding predicate $h(r, s, U)$ producing mutually exclusive assignments. This property is given for any consistent hashing function, as described in *Section 2.1*.

The non-trivial case is one in which one or more peers observe a change in the fleet while the shard is still in state *Steady* and want to take ownership of resources currently assigned to the shard. To prevent peers from taking over resources still being reconciled by the shard, a two-phase takeover procedure is used, similar to the existing solutions of Ebert and the multicluster runtime. The first phase is achieved through the *drain* action: Shards change their predicate to reconcile only resources assigned under both the previously and the newly observed fleet. The second phase, achieved

through the *expand* and *stabilize* actions, only runs once peers acknowledge that they have successfully executed the *drain* action.

Once the shard observes the changed fleet, the described *drain* action is run. Once the updated predicate is active, the incremented revision version is published to acknowledge the new state to the peers. The shard is now in state *Waiting*, waiting for its peers to acknowledge their new state as well.

After transitioning from *Steady* to *Waiting*, mutual exclusion is guaranteed because each shard reconciles only a subset of its previously assigned resources.

If a change of the fleet is observed while in state *Waiting*, the same procedure is repeated and mutual exclusion is guaranteed for the same reason: *drain* only reduces the predicate to a subset of the previously assigned resources and never leads to newly claimed ownership.

Once the shard observes one or more peers with a revision version higher than or equal to its own (i.e. peers that are *caught up*), the second phase runs. The *expand* action is run if Δ_s is not empty but not all peers are caught up yet. Because caught-up peers have acknowledged that they stopped reconciling resources no longer assigned to them, the shard can safely start reconciling resources transitioning from caught-up peers to itself. Mutual exclusion is preserved.

The same holds for the *stabilize* action, which runs once all peers have caught up. *stabilize* results in the same predicate as calling *expand* for all peers and therefore ensures mutual exclusion as well. After *stabilize*, the shard does not have to wait for any peers to catch up and therefore returns to state *Steady*.

The edge case for both *stabilize* and *expand* is a fleet change during or after calling the action. In such a case, mutual exclusion is maintained by the previously described mechanism: peers do not take ownership of resources until the shard acknowledges the new revision after the *drain* procedure.

This covers all possible situations of the state machine and shows that mutual exclusion is guaranteed at any point in time by the revision control procedure.

However, one edge case cannot be covered by this system. If a garbage collector pause or system clock skew causes the lease manager to miss a lease renewal, a race condition can cause concurrent reconciles of the same resource. In the time between the lease expiring and the lease manager triggering an *operator* shutdown, mutual exclusion is not guaranteed. The likelihood of this occurring can be reduced by using a long lease time-to-live and a short renewal interval, giving the lease manager enough time to recover from delayed renewals.

Liveness

In addition to mutual exclusion, a fleet must eventually converge on state *Steady* with a stable *fleet*. The mutual exclusion explanation above already shows the normal path of the two-step handover protocol; there are, however, edge cases that must also be handled by the procedure. The problem can be narrowed down to the following: every *shard* must eventually observe every peer's incremented revision counter so that the *Stabilize* precondition becomes true, triggering a transition to *Steady*.

The first edge case is an unresponsive *shard* that consequently never acknowledges the new *fleet*. This keeps *shards* in the *Waiting* state until the lease expires or the *shard* becomes responsive again, ensuring liveness after at most the time-to-live of the lease that the unresponsive *shard* never renews.

Because *shards* need to await reconciles of resources no longer assigned during *drain*, a second edge case is a stuck reconcile causing a *shard* to never finish the *drain* procedure. By default, this edge case is not handled and can cause a fleet to remain in a *Waiting* state forever. Due to the *expand* transition, resources of remaining *shards* can still be transitioned, ensuring fleet responsiveness. If this case is a concern, a `ReconciliationTimeout` can be set on controllers that limits the maximum duration of a reconcile.

6 Evaluation

This chapter evaluates the architecture from *Section 5* against the evaluation objectives defined in *Section 4.2* and thereby answers *RQ 2*. Unless specified otherwise, benchmarks use the website operator introduced in *Section 4.2*, which reconciles website resources.

Load Fairness

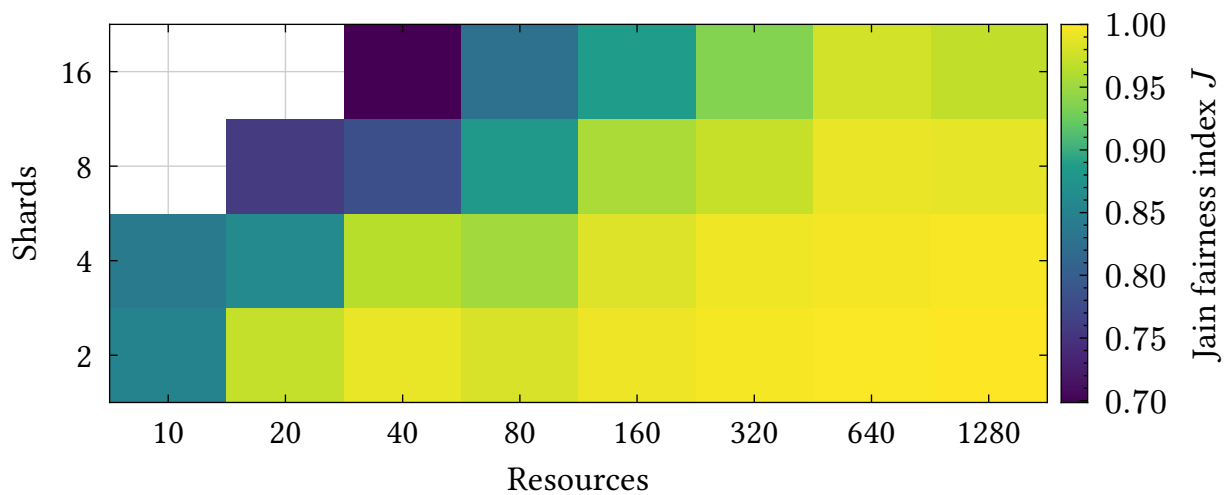


Figure 15: Jain fairness index J for different shard and resource counts, computed from reconcile times over 60 s. Cells with fewer than two resources per shard are omitted, both because their fairness is expected to be low and to increase the color resolution of the remaining cells. Higher is better.

The benchmark measuring Jain’s fairness index quantifies fairness across different configurations of shard and resource counts, as required by *EO 1*. Website resources are created with random values for the reconcile interval (uniformly drawn from $[0, 15] \in \mathbb{N}$ seconds) and call an endpoint that allows specifying the response time, which is randomized in the same way (from $[0, 10] \in \mathbb{N}$ seconds). This allows the benchmark to simulate heterogeneous input with resources causing different amounts of reconcile utilization. The shards are started once the resources have been created, and the reconcile time is measured over 120 seconds. *Figure 15* shows the measured Jain fairness index for two to 16 shards and ten to 1280 resources. Three cells with fewer than two resources per shard are omitted due to their expected low fairness (as low as 0.37) to preserve color resolution for the remaining cells. As the number of resources

grows, fairness increases rapidly; more shards result in lower fairness. However, at 1280 resources, even 16 shards achieve fairness above 95%. Because sharded operators are typically only used with thousands of resources, a near-perfect distribution can be expected.

Memory Overhead

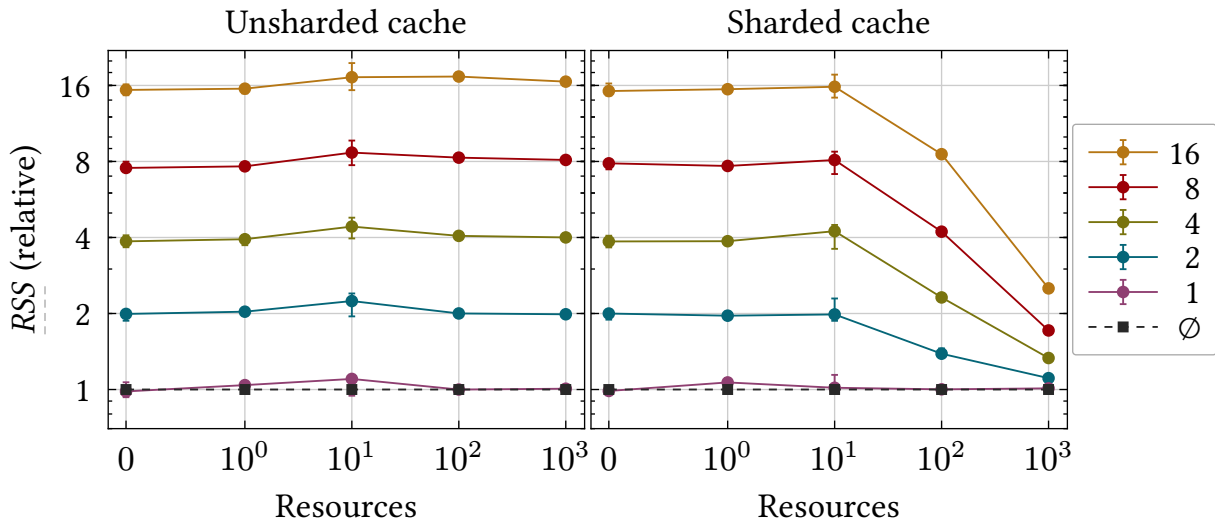


Figure 16: Relative RSS against resource count for different shard counts, with $\emptyset$ marking the unsharded baseline. Each run reports the median RSS over the last 20 s of a 70 s run; points are medians across runs and whiskers show min/max. Left: unsharded cache; right: sharded cache. Lower is better.

To measure memory overhead according to *EO 2*, website resources are created with 100 kilobytes of additional dummy data to increase the memory footprint of cached resources. After the resources have been created, the shards are started, and after settling for 50 seconds, the median RSS over 20 seconds is measured at one-second intervals. The benchmark is executed for an unsharded operator that does not use *kube-sharder* and for sharded operators from one to 16 shards, each with and without the sharded cache enabled. Figure 16 shows the two resulting diagrams, one with the sharded cache and one without, both relative to the unsharded operator. Without the sharded cache enabled, a fleet has memory overhead proportional to the number of shards, regardless of the number of resources. Enabling the sharded cache does not make a difference at first, but with more resources, the relative memory overhead quickly shrinks.

The measurements clearly show that *kube-sharder* itself does not impose significant memory overhead. A *fleet* with a single *shard* requires just as much memory as an unsharded *operator*. For a small cache, memory overhead scales closely with the number of *shards*, which is expected because all shards start the same informers and allocate the same objects. As the cache grows, the size of these shared objects becomes less significant. With a sharded cache, each *shard* only stores relevant resources, greatly reducing memory overhead.

For an *operator* reconciling thousands of resources, the cache will always dominate memory usage. The sharded cache can therefore reduce the overhead to a minimum, making adoption even easier. Ebert achieves similar memory overhead using the resource-label approach but does not support cache sharding for resource types watched with the `Watch` function of the controller builder; only `For` and `Owns` resources are supported.

CPU Overhead

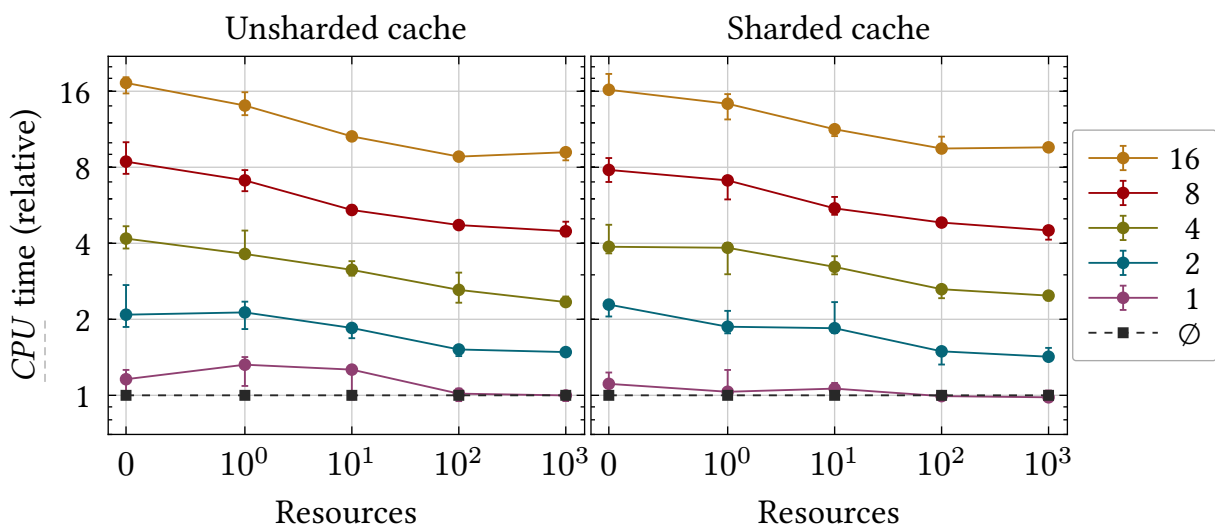


Figure 17: Relative CPU time against resource count for different shard counts, with $\emptyset$ marking the unsharded baseline. Measured over 60 s at a 10 s reconcile interval. Left: unsharded cache; right: sharded cache. Points are medians across runs; whiskers show min/max. Lower is better.

CPU time is measured similarly to utilization fairness. Website resources are created that reconcile every 10 seconds and request an endpoint. CPU time is measured over

60 seconds with a 10-second warmup once the *shards* have started. As in the memory overhead benchmark, an unsharded *operator* is used as the baseline for comparison, and *shards* are tested with and without the sharded cache enabled. Figure 17 shows that *fleets* have overhead corresponding to the number of *shards* at zero resources. With more resources, the relative *CPU* time slightly decreases but remains significant.

At zero resources, overhead matching the *shard* count is expected, as the baseline computations are simply replicated across multiple instances. The diagrams also show no significant difference between the sharded and unsharded cache, despite the added complexity of the components described in Section 5.4.

The relative *CPU* usage remaining significant even with many resources is misleading. The reconcile function of the website *operator* merely calls an *API* endpoint and therefore results in minimal *CPU* usage. While the relative *CPU* usage is high, at more than eight times the baseline with 16 *shards*, the overall *CPU* usage remains very low.

Figure 18 confirms this. The diagram shows the result of a benchmark where 1000 resources are reconciled at a 10-second interval by different *fleet* sizes. Instead of calling an *Hypertext Transfer Protocol (HTTP)* endpoint, the reconcile function computes

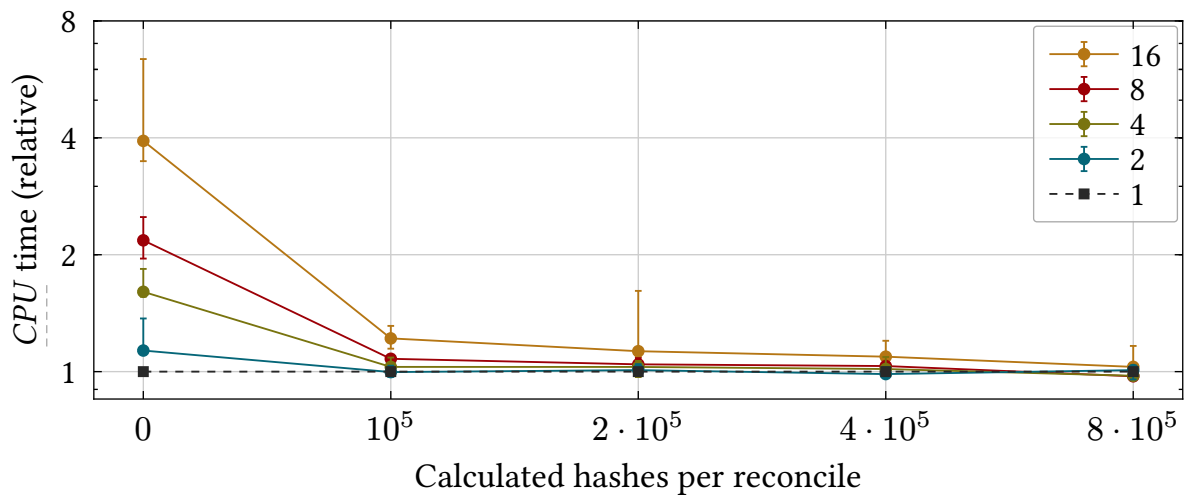


Figure 18: Relative *CPU* time against per-reconcile compute cost for different *shard* counts, with the single-shard variant as baseline. Compute cost is simulated by computing SHA-256 hashes per reconcile. Measured over 60 s at a 10 s reconcile interval, sharded cache enabled. Points are medians across runs; whiskers show min/max. Lower is better.

a configurable number of SHA-256 hashes. The maximum of $8 \cdot 10^5$ hashes per reconcile was chosen because it nearly saturates the benchmarking machine. This makes the upper end of the benchmark dominated by actual reconcile work rather than by fixed sharding overhead, and therefore shows whether the relative overhead converges towards the single-shard baseline as reconcile cost increases.

When calculating zero hashes, the relative CPU time reflects the results of *Figure 17*; however, as the number of hashes per reconcile grows, the relative CPU time quickly collapses to less than a factor of 1.25.

Overall, this shows that the CPU overhead behaves similarly to the memory overhead: at low CPU usage, the relative usage closely follows the fleet size, but as the baseline CPU usage grows, the relative usage quickly converges to that of the baseline implementation.

API Server Request Overhead

To determine request overhead as described in *EO 4*, a benchmark was created that creates resources and then starts modified operators that log every API request, using sharded operators with the sharded cache enabled. The created website resources are reconciled at a 10-second interval, and during a reconcile, the status subresource is updated, resulting in one write request to the API Server per reconcile. The benchmark runs for 60 seconds per measurement; *Figure 19* and *Figure 20* show the results. *Figure 19* shows behavior similar to previous benchmarks: at low resource usage, *kube-sharder* results in very high relative API request overhead. This is due to the leases maintained by the fleet and the replicated informers running in each shard. However, as the resource count grows, the overhead quickly diminishes, approaching one at 1000 resources. *Figure 20* shows the distribution of request kinds at 1000 resources. For all variants, most requests are edit requests from status updates during reconciles and from lease renewals. About 7.5% of requests are get requests for the sharded variants, which also account for most of the overhead. These get requests come from the cache resync workflow on startup, as described in *Section 5.4*. List and watch requests appear only as traces, making up at most 0.7% at 16 shards.

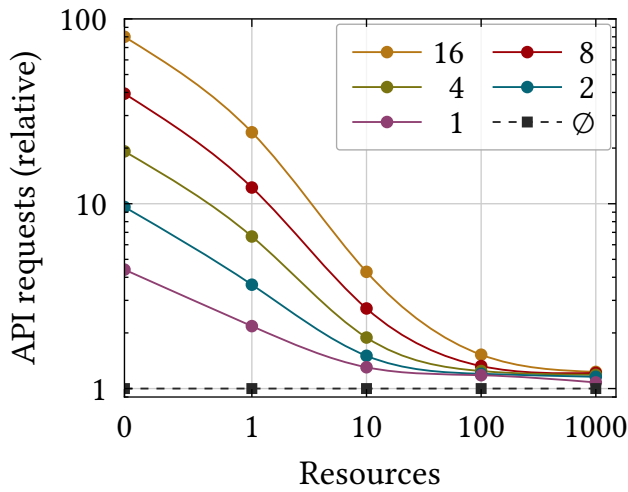


Figure 19: Relative API request count against resource count for different shard counts, with $\emptyset$ marking the unsharded baseline. Requests counted over 120 s while reconciling every web-site resource every 10 s with one status write per reconcile. Sharded cache enabled. Lower is better.

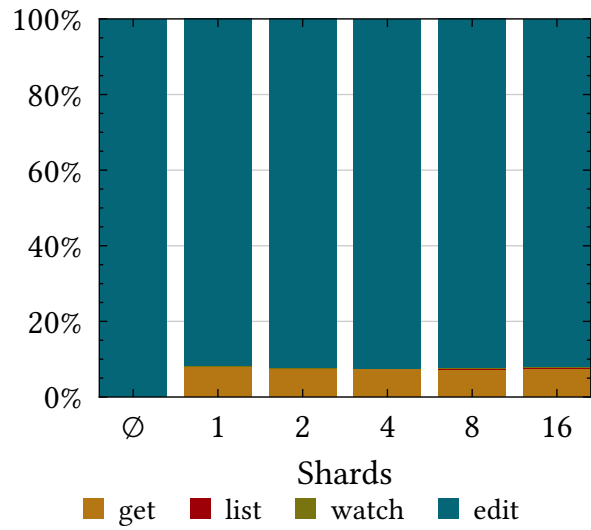


Figure 20: Breakdown of API request types by shard count at 1000 resources. Same benchmark as Figure 19.

This benchmark clearly shows that request-count overhead is negligible at scale, including the case where the sharded cache is enabled. Within a short benchmark window of 60 seconds, the initial get requests make up only a small portion; for fleets that remain stable over long periods, this initial sync during membership churn becomes increasingly insignificant. Without the sharded cache, the get requests are omitted at the cost of higher memory usage, as shown earlier, reducing API request overhead further.

Compared to the existing solutions of Ebert and the multicluster runtime, this is significantly lower request overhead. During membership churn, kube-sharder makes at most one get request per transitioned resource to update the cache, plus one edit request per shard to publish the incremented revision version. Ebert's solution results in two write requests per transitioned resource. The multicluster runtime, in addition to two

write requests per transitioning resource during *membership churn*, also maintains a lease per resource, resulting in even more write requests.

Storage Overhead

EO 5 is measured by calculating the *YAML* size of all resources or changes added by *kube-sharder*, which amounts to merely one lease resource per *shard*. The size of the created lease depends on the length of the shard and *fleet* names. For a shard name of about 60 characters and a *fleet* name of about 20 characters, the lease object has a size of around 600 bytes, resulting in a storage overhead of up to 600 bytes per *shard*. This overhead is negligible and similar to Ebert's solution, which also maintains a lease per *shard*. In addition, Ebert's solution adds a label to each reconciled resource, and the multicluster runtime creates an additional lease per resource. While these solutions therefore have a theoretically higher storage overhead, their overhead is negligible as well, since additional labels and leases are very small objects, as shown above.

Reconcile Latency

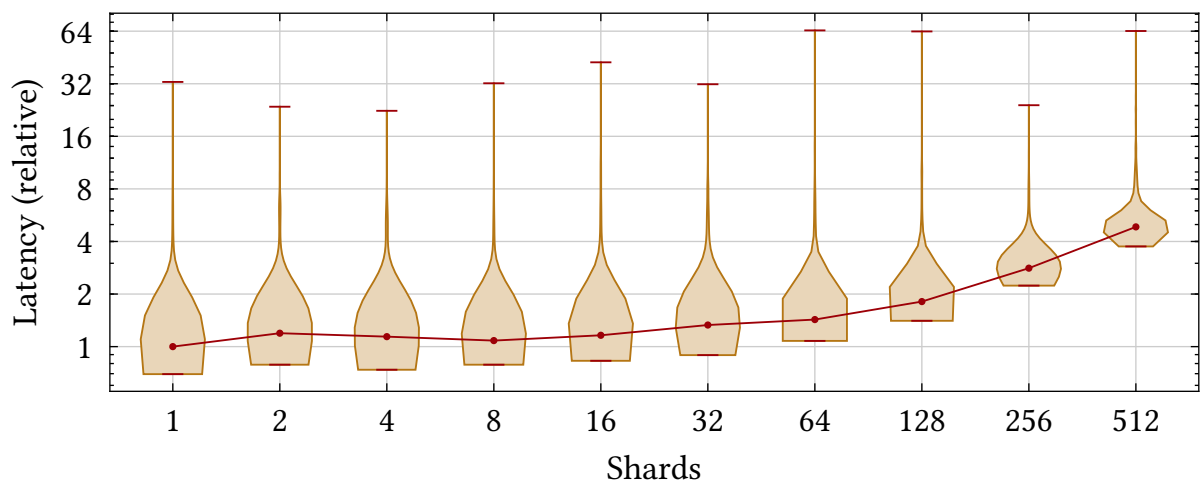


Figure 21: Relative reconcile latency against *shard* count, with the unsharded variant as baseline (constant 1, omitted from the plot). Latency is the delay from an *API* event reaching the operator's informer to the reconcile function being invoked. Points are medians across runs; violins show the sample distribution with min/max extrema marked. Sharded cache enabled. Lower is better.

The reconcile latency describes the time it takes for a reconcile event to travel from arriving at the informer to the reconcile function being called, as described in EO 6. To

measure this, the sharded (with sharded cache enabled) and unsharded website operators were modified to expose an endpoint that allows sending an artificial `API` event to an informer. The benchmark then sends an artificial event and measures the time until the reconcile function calls back the benchmarking script. *Figure 21* shows the result relative to the unsharded operator as baseline. The violins visualize the distribution of latencies. With growing `fleet` size, relative latency grows visibly from 32 `shards` onwards; for `fleet` sizes of 16 and below, latency stays close to or slightly above one.

The growing latency is expected due to the linear scaling of `HRW` compute time, the consistent hashing algorithm used. However, the graph shows that a `fleet` size in the range of tens does not increase reconcile latency significantly.

If hundreds of `shards` are desired, an alternative consistent hashing algorithm that runs in constant time, such as Karger consistent hashing, could be preferred. As `kube-sharder` allows swapping the consistent hashing algorithm as described in *Section 5*, switching to Karger consistent hashing can be done without modifying the library source code.

Failover Time

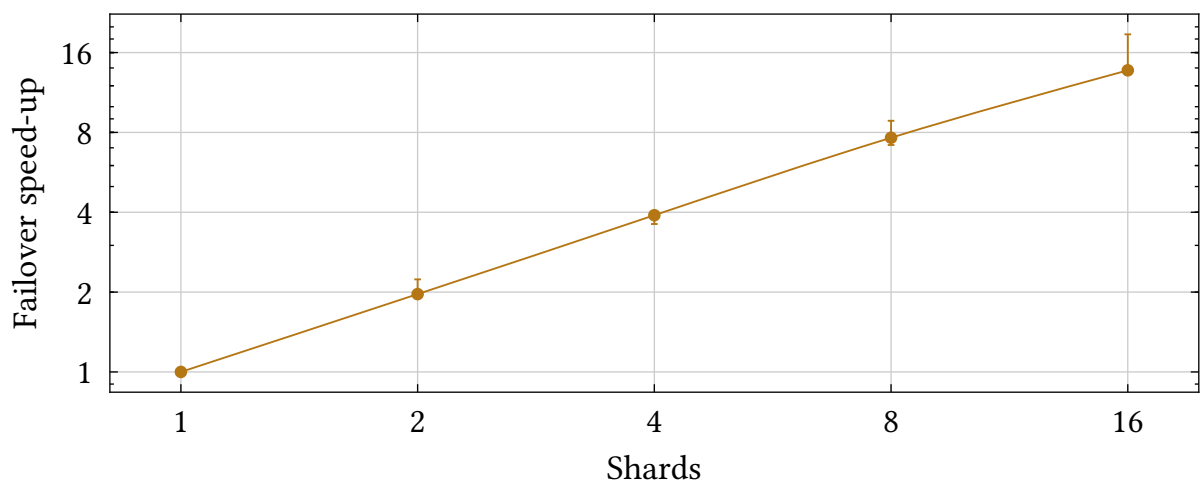


Figure 22: Median failover speed-up against `shard` count, with the single-shard variant as baseline. Failover speed is measured as the time all `shard` work queues take to drain the reconcile burst triggered by replacing one operator pod. Points are medians across runs; whiskers show min/max.

Benchmark: 1024 resources, each reconcile taking 1 s. Higher is better.

Finally, *EO 7* describes the failover-time metric as the time it takes remaining and replacing *shards* to reconcile the transitioned resources of a deleted *shard*. This was measured by modifying the sharded *operator* to record and expose the time from startup until the initial burst of reconcile requests is drained. The benchmark created 1024 resources with reconciles taking one second and no concurrent reconciles allowed per *shard*. Once the started *shards* had drained the initial reconcile request burst caused by startup, a *shard* was deleted and the measurement described above was awaited.

Figure 22 shows the increasing failover speed-up for a growing *fleet* size. The failover speed-up approximately matches the *fleet* size.

This observation follows the expected behavior. As described in *Section 2.1*, with high probability $\frac{r}{s}$ resources are reassigned when a *shard* is deleted, for r resources and s shards. Failover time is therefore reduced by a factor of $\frac{1}{s}$, i.e. sped up by s .

Existing solutions leveraging consistent hashing for resource assignments, such as Ebert and the multicluster runtime, achieve the same failover speed-up.

7 Conclusion

An *operator* built with *Kubebuilder* is bound by its single-leader design. Under leader election, only one replica is active at a time, so both *CPU* and memory footprint scale with the number of managed resources but remain bounded by a single *node*. Additional replicas do not distribute load; they remain idle as followers. Failover behaves the same way: after leader failure, *controller-runtime*'s informer sync enqueues an event for every existing object, so the new leader must reconcile every managed resource once before it catches up. Both effects share the same root cause and both grow linearly with the number of managed resources. The analysis in *Section 3* shows that no existing horizontal-scaling approach provides *minimal disruption*, *monotonicity*, mutual exclusion, low *API Server* load, and transparent *Kubebuilder* integration at the same time. Each of the five surveyed solutions is missing at least one of these properties, as summarized in *Table III*.

This thesis presents *kube-sharder*, a drop-in *Kubebuilder* extension that closes this gap. Work is distributed across replicas using *HRW* consistent hashing. Mutual exclusion during *membership churn* is provided by the revision control state machine described in *Section 5.5.4*, a lease-based handover protocol that communicates only through the *API Server*. Memory usage can additionally be reduced using the sharded cache described in *Section 5.4*. All coordination goes through *Kubernetes* leases, so no external component beyond the sharded *operator* itself is required. This answers *RQ 1*: existing reconcilers work unmodified through the `ShardedControllerBuilder`, which mirrors the *controller-runtime* builder *API*. *Minimal disruption* follows from the chosen consistent hashing algorithm, and mutual exclusion is guaranteed by the revision control protocol and the wrapped work queue. The correctness of the revision control protocol is discussed in *Section 5.5.7*.

Section 6 quantifies the resulting overhead and thereby answers *RQ 2*. At *fleet* sizes of practical interest, the Jain fairness index exceeds 0.95 at 1280 resources across 16 *shards* (*Figure 15*), so load is distributed close to optimal once the resource count reaches the thousands. With the sharded cache enabled, memory overhead relative to the unsharded baseline drops close to zero as the cache dominates the *operator*'s footprint (*Figure 16*). The *API Server* request overhead approaches zero at the same scale (*Figure 19*). Failover time is reduced by a factor equal to the *fleet* size (*Figure 22*),

matching the r/s reassignment bound of consistent hashing established in *Section 2.1*. Compared to the closest prior work, Ebert’s label-based sharder, *kube-sharder* reaches the same memory and failover behavior without a dedicated external *sharding* deployment or additional *CRs*, and makes at most one get request per transitioned resource during *membership churn* instead of two write requests per transition.

Two limitations restrict this contribution and motivate the next steps. First, the implementation is restricted to *operators* built with *Kubebuilder*. *Operators* written with other frameworks or in other programming languages are not supported. Second, all benchmarks in *Section 6* use the synthetic website *operator* introduced in *Section 4.2*, whose reconcile cost is either a single *HTTP* request or a fixed number of SHA-256 hashes. Real reconcile-cost distributions and event patterns may differ from these controlled profiles.

Both limitations mark directions for future work. Parts of the *kube-sharder* design are framework-agnostic and can be reused to implement sharding libraries for other frameworks and programming languages. Load-aware sharding would let the consistent hashing function redistribute resources based on the observed reconcile cost of individual *shards* instead of on resource identity alone. This would reduce the gap between synthetic uniform workloads and real production ones, and would also support heterogeneous *fleets* of *shards* with different resource capacities. Consistent hashing algorithms already support this case in principle, as shown in *Section 2.1*, but it is not yet implemented in the library. Additional consistent hashing algorithms, in particular DxHash, could be offered as an opt-in for *operators* deployed as a `StatefulSet`, where absolute *shard* numbering is available and the near-constant lookup cost of DxHash becomes useful for large *fleets*, as discussed in *Section 5*. Upstreaming the sharded manager, monitored work queue, and revision control components into *controller-runtime* itself would further reduce adoption cost, since *operator* authors would no longer need to depend on a separate library. Beyond engineering work, a more in-depth evaluation on real workloads would help validate the synthetic benchmark numbers against real-world reconcile-cost distributions. Formal verification of the revision control protocol under *membership churn*, for example in TLA+¹⁹, would raise the correctness argument of *Section 5.5.7* from prose to a machine-checked proof.

¹⁹<https://lamport.azurewebsites.net/tla/tla.html>

Bibliography

- [1] The Kubernetes Authors, “The Kubebuilder Book.” Accessed: Jun. 29, 2026. [Online]. Available: <https://book.kubebuilder.io/>
- [2] D. Karger, E. Lehman, T. Leighton, R. Panigrahy, M. Levine, and D. Lewin, “Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web,” in *Proceedings of the Twenty-Ninth Annual ACM Symposium on Theory of Computing (STOC '97)*, ACM, 1997, pp. 654–663. doi: [10.1145/258533.258660](https://doi.org/10.1145/258533.258660).
- [3] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. Sebastopol, CA: O'Reilly Media, 2017.
- [4] I. Stoica *et al.*, “Chord: a scalable peer-to-peer lookup protocol for internet applications,” *IEEE/ACM Transactions on Networking*, vol. 11, no. 1, pp. 17–32, 2003, doi: [10.1109/TNET.2002.808407](https://doi.org/10.1109/TNET.2002.808407).
- [5] D. R. Karger and M. Ruhl, “Simple Efficient Load Balancing Algorithms for Peer-to-Peer Systems,” in *Proceedings of the 16th ACM Symposium on Parallelism in Algorithms and Architectures (SPAA '04)*, 2004, pp. 36–43. doi: [10.1145/1007912.1007919](https://doi.org/10.1145/1007912.1007919).
- [6] G. DeCandia *et al.*, “Dynamo: Amazon's Highly Available Key-value Store,” in *Proceedings of 21st ACM SIGOPS Symposium on Operating Systems Principles (SOSP '07)*, ACM, 2007, pp. 205–220. doi: [10.1145/1294261.1294281](https://doi.org/10.1145/1294261.1294281).
- [7] D. G. Thaler and C. V. Ravishankar, “A Name-Based Mapping Scheme for Rendezvous,” technical report CSE-TR-316-96, 1996.
- [8] D. G. Thaler and C. V. Ravishankar, “Using Name-Based Mappings to Increase Hit Rates,” *IEEE/ACM Transactions on Networking*, vol. 6, no. 1, pp. 1–14, 1998, doi: [10.1109/90.663936](https://doi.org/10.1109/90.663936).
- [9] W. Wang and C. V. Ravishankar, “Hash-Based Virtual Hierarchies for Scalable Location Service in Mobile Ad-hoc Networks,” *Mobile Networks and Applications*, vol. 14, no. 5, pp. 625–637, 2009, doi: [10.1007/s11036-008-0144-3](https://doi.org/10.1007/s11036-008-0144-3).
- [10] V. Valloppillil and K. W. Ross, “Cache Array Routing Protocol v1.0.” [Online]. Available: <https://datatracker.ietf.org/doc/html/draft-vinod-carp-v1-03>

- [11] J. Resch, “New Hashing Algorithms for Data Storage.” [Online]. Available: https://www.snia.org/sites/default/files/SDC15_presentations/dist_sys/Jason_Resch_New_Consistent_Hashings_Rev.pdf
- [12] C. Dong, F. Wang, and D. Feng, “DxHash: A Memory-saving Consistent Hashing Algorithm,” *ACM Transactions on Internet Technology*, vol. 24, no. 1, p. 3:1–3:22, 2024, doi: [10.1145/3631708](https://doi.org/10.1145/3631708).
- [13] M. Kleppmann, “How to do distributed locking.” Accessed: Jun. 09, 2026. [Online]. Available: <https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html>
- [14] M. Burrows, “The Chubby lock service for loosely-coupled distributed systems,” in *7th USENIX Symposium on Operating Systems Design and Implementation (OSDI '06)*, USENIX Association, 2006, pp. 335–350.
- [15] etcd Authors, “etcd API.” Accessed: Jun. 17, 2026. [Online]. Available: <https://etcd.io/docs/v3.6/learning/api/>
- [16] K. Kingsbury, “Jepsen: etcd 3.4.3.” Accessed: Jun. 09, 2026. [Online]. Available: <https://jepsen.io/analyses/etcd-3.4.3>
- [17] U. Joshi, *Patterns of Distributed Systems*. in Addison-Wesley Signature Series (Fowler). Addison-Wesley Professional, 2024.
- [18] L. Lamport, “Time, clocks, and the ordering of events in a distributed system,” *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978, doi: [10.1145/359545.359563](https://doi.org/10.1145/359545.359563).
- [19] D. Ongaro and J. Ousterhout, “In Search of an Understandable Consensus Algorithm,” in *2014 USENIX Annual Technical Conference (USENIX ATC '14)*, USENIX Association, 2014, pp. 305–319. [Online]. Available: <https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro>
- [20] P. Hunt, M. Konar, F. P. Junqueira, and B. Reed, “ZooKeeper: Wait-free Coordination for Internet-scale Systems,” in *Proceedings of the 2010 USENIX Annual Technical Conference (USENIX ATC '10)*, USENIX Association, 2010. [Online]. Available: https://www.usenix.org/legacy/events/atc10/tech/full_papers/Hunt.pdf
- [21] F. B. Schneider, “Implementing Fault-Tolerant Services Using the State Machine Approach: A Tutorial,” *ACM Computing Surveys*, vol. 22, no. 4, pp. 299–319, 1990, doi: [10.1145/98163.98167](https://doi.org/10.1145/98163.98167).

- [22] L. Lamport, D. Malkhi, and L. Zhou, “Vertical Paxos and Primary-Backup Replication,” in *Proceedings of the 28th ACM Symposium on Principles of Distributed Computing (PODC '09)*, ACM, 2009, pp. 312–313. doi: [10.1145/1582716.1582783](https://doi.org/10.1145/1582716.1582783).
- [23] M. Lukša and K. Conner, *Kubernetes in Action*, 2nd ed. Shelter Island, NY: Manning Publications, 2024.
- [24] S. Deng *et al.*, “Cloud-Native Computing: A Survey from the Perspective of Services.” [Online]. Available: <https://arxiv.org/abs/2306.14402>
- [25] The Linux Foundation, “New Cloud Native Computing Foundation to Drive Alignment Among Container Technologies.” [Online]. Available: <https://www.cncf.io/announcements/2015/06/21/new-cloud-native-computing-foundation-to-drive-alignment-among-container-technologies/>
- [26] C. Aniszczyk, “[RESULT] Kubernetes Vote by TOC – Kubernetes Accepted as First Incubated Project of the Cloud Native Computing Foundation.” [Online]. Available: https://lists.cncf.io/g/cncf-toc/topic/result_kubernetes_vote_by/8008089
- [27] Cloud Native Computing Foundation and Linux Foundation Research, “CNCf Annual Cloud Native Survey 2025: Kubernetes as the De Facto Operating System for AI,” technical report, Jan. 2026. [Online]. Available: <https://www.cncf.io/announcements/2026/01/20/kubernetes-established-as-the-de-facto-operating-system-for-ai-as-production-use-hits-82-in-2025-cncf-annual-cloud-native-survey/>
- [28] Gartner, Inc., “Magic Quadrant for Container Management,” technical report, 2025. [Online]. Available: <https://aws.amazon.com/resources/analyst-reports/gartner/magic-quadrant-for-container-management/>
- [29] E. Tune, “Growth and Design Patterns in the Extensions Ecosystem.” [Online]. Available: <https://kccncna19.sched.com/event/UaYw>
- [30] The Kubernetes Authors, “Kubernetes API Concepts.” Accessed: Jun. 11, 2026. [Online]. Available: <https://kubernetes.io/docs/reference/using-api/api-concepts/>
- [31] The Kubernetes Authors, “Custom Resources.” Accessed: Jun. 15, 2026. [Online]. Available: <https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/custom-resources/>

- [32] OpenAPI Initiative, “OpenAPI Specification, Version 3.0.0.” Accessed: Jun. 15, 2026. [Online]. Available: <https://github.com/OAI/OpenAPI-Specification/blob/3.0.0/versions/3.0.0.md>
- [33] The Kubernetes Authors, “Extend the Kubernetes API with CustomResourceDefinitions.” Accessed: Jun. 15, 2026. [Online]. Available: <https://kubernetes.io/docs/tasks/extend-kubernetes/custom-resources/custom-resource-definitions/>
- [34] The Kubernetes Authors, “Dynamic Admission Control.” Accessed: Jun. 15, 2026. [Online]. Available: <https://kubernetes.io/docs/reference/access-authn-authz/extensible-admission-controllers/>
- [35] The Kubernetes Authors, “Cluster Architecture.” Accessed: Jun. 16, 2026. [Online]. Available: <https://kubernetes.io/docs/concepts/architecture/>
- [36] The Kubernetes Authors, “Authenticating.” Accessed: Jun. 16, 2026. [Online]. Available: <https://kubernetes.io/docs/reference/access-authn-authz/authentication/>
- [37] The Kubernetes Authors, “Authorization.” Accessed: Jun. 16, 2026. [Online]. Available: <https://kubernetes.io/docs/reference/access-authn-authz/authorization/>
- [38] Kubernetes SIG-Architecture, “Kubernetes API Conventions.” [Online]. Available: <https://github.com/kubernetes/community/blob/master/contributors/devel/sig-architecture/api-conventions.md>
- [39] The Kubernetes Authors, “kube-apiserver.” Accessed: Jun. 17, 2026. [Online]. Available: <https://kubernetes.io/docs/reference/command-line-tools-reference/kube-apiserver/>
- [40] etcd Authors, “etcd API guarantees.” Accessed: Jun. 17, 2026. [Online]. Available: https://etcd.io/docs/v3.6/learning/api_guarantees/
- [41] Kubernetes SIG API Machinery, “KEP-4568: Resilient watchcache initialization.” Accessed: Jun. 18, 2026. [Online]. Available: <https://github.com/kubernetes/enhancements/blob/996ec172835bb4b3653332cd08c6c65fea4919b2/keps/sig-api-machinery/4568-resilient-watchcache-initialization/README.md>
- [42] The Kubernetes Authors, “API Priority and Fairness.” Accessed: Jun. 18, 2026. [Online]. Available: <https://kubernetes.io/docs/concepts/cluster-administration/flow-control/>

- [43] etcd Authors, “System limits – etcd.” Accessed: Jun. 18, 2026. [Online]. Available: <https://etcd.io/docs/v3.6/dev-guide/limit/>
- [44] Kubernetes SIG Scalability, “Kubernetes Scalability thresholds.” Accessed: Jun. 18, 2026. [Online]. Available: <https://github.com/kubernetes/community/blob/main/sig-scalability/configs-and-limits/thresholds.md>
- [45] The Kubernetes Authors, “API Machinery Special Interest Group.” Accessed: Jun. 18, 2026. [Online]. Available: <https://github.com/kubernetes/community/blob/main/sig-api-machinery/README.md>
- [46] The Flux authors, “Flux sharding and horizontal scaling.” Accessed: Jul. 08, 2026. [Online]. Available: <https://fluxcd.io/flux/installation/configuration/sharding/>
- [47] KubeVela Contributors, “Application Controller Sharding (design/vela-core/sharding.md).” Accessed: Jul. 08, 2026. [Online]. Available: <https://github.com/kubevela/kubevela/blob/master/design/vela-core/sharding.md>
- [48] T. Ebert, “Horizontally Scalable Kubernetes Controllers,” Master's thesis, 2024.
- [49] Argo Project, “Argo CD – Dynamic Cluster Distribution.” Accessed: Jul. 09, 2026. [Online]. Available: <https://argo-cd.readthedocs.io/en/stable/operator-manual/dynamic-cluster-distribution/>
- [50] ivan-cai, “Dynamic Cluster Distribution: some applications not be processed when shards scale down, and some applicaitons will be process by more than one shard while scaling up.” Accessed: Jul. 09, 2026. [Online]. Available: <https://github.com/argoproj/argo-cd/issues/24520>
- [51] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design Science in Information Systems Research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004, Accessed: Feb. 15, 2025. [Online]. Available: <http://www.jstor.org/stable/25148625>
- [52] “ISO/IEC/IEEE International Standard - Systems and software engineering – Life cycle processes – Requirements engineering,” *ISO/IEC/IEEE 29148:2018(E)*, vol. , no. , pp. 1–104, 2018, doi: [10.1109/IEEESTD.2018.8559686](https://doi.org/10.1109/IEEESTD.2018.8559686).
- [53] A. Mavin, P. Wilkinson, A. Harwood, and M. Novak, “Easy approach to requirements syntax (EARS),” Oct. 2009, pp. 317–322. doi: [10.1109/RE.2009.9](https://doi.org/10.1109/RE.2009.9).

- [54] V. R. Basili, G. Caldiera, and H. D. Rombach, “The Goal Question Metric Approach,” in *Encyclopedia of Software Engineering*, vol. 1, J. J. Marciniak, Ed., John Wiley & Sons, 1994, pp. 528–532.
- [55] J. F. Nunamaker, M. Chen, and T. D. M. Purdin, “Systems Development in Information Systems Research,” *Journal of Management Information Systems*, vol. 7, no. 3, pp. 89–106, 1990, Accessed: Jun. 18, 2026. [Online]. Available: <http://www.jstor.org/stable/40397957>
- [56] H. A. Simon, *The sciences of the artificial*, 3rd ed. in The MIT Press. London, England: MIT Press, 1996.
- [57] W3C, “High Resolution Time.” [Online]. Available: <https://www.w3.org/TR/hr-time/>
- [58] R. Cox, “Proposal: Monotonic Elapsed Time Measurements in Go.” [Online]. Available: <https://go.gosourcelab.com/proposal/+/master/design/12914-monotonic.md>
- [59] Linux Kernel Documentation, “Control Group v2.” [Online]. Available: <https://docs.kernel.org/admin-guide/cgroup-v2.html>
- [60] R. Jain, D.-M. Chiu, and W. R. Hawe, “A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems,” technical report DEC-TR-301, Sep. 1984. Accessed: Jul. 30, 2026. [Online]. Available: https://ocw.cs.pub.ro/courses/_media/isrm/laboratoare/new/a_quantitative_measure_of_fairness_and_d.pdf
- [61] M. Gorman, *Understanding the Linux Virtual Memory Manager*. Upper Saddle River, NJ: Prentice Hall, 2004.
- [62] J. Tibshirani, “Beyond Working Set Memory: Understanding the cAdvisor Memory Metrics.” [Online]. Available: <https://sourcegraph.com/blog/beyond-working-set-memory-understanding-the-cadvisor-memory-metrics>
- [63] C. Coleman, “Protobuf serialization and internal storage.” Accessed: Jul. 10, 2026. [Online]. Available: <https://github.com/kubernetes/design-proposals-archive/blob/main/api-machinery/protobuf.md>
- [64] N. A. Lynch and M. R. Tuttle, “An Introduction to Input/Output Automata,” *CWI Quarterly*, vol. 2, no. 3, pp. 219–246, 1989.

Abbreviations

API – Application Programming Interface

CLI – Command Line Interface

CNCF – Cloud Native Computing Foundation

CPU – Central Processing Unit

CRUD – create, read, update, and delete

EARS – Easy Approach to Requirements Syntax

GQM – Goal Question Metric

HA – high availability

HRW – Highest Random Weight

HTTP – Hypertext Transfer Protocol

PRNG – Pseudo-Random Number Generator

REST – Representational State Transfer

RSS – Resident Set Size

SLO – Service Level Objective

YAML – Yet Another Markup Language

Kubernetes

CR – Custom Resource

CRD – Custom Resource Definition

GVK – Group Version Kind

KRM – Kubernetes Resource Model

Glossary

Kubernetes: Kubernetes is an open-source container orchestration platform that automates the deployment, scaling, and management of containerized applications across clusters of machines. *Section 2.3* explains in more detail.

Kubebuilder: Kubebuilder is a Go-based framework for building Kubernetes APIs and operators using CRDs, providing scaffolding tools and a controller runtime that abstracts away the complexity of interacting with the Kubernetes API server. *Section 2.4* explains in more detail.

Distributed Systems Foundations

fencing token: Monotonically increasing token attached to a lock, lease, or resource. Recipients check the token against the highest one they have observed and reject requests carrying an older token, preventing stale lock holders from performing unsafe writes. *Section 2.2* explains in more detail.

fleet: The set of currently running shards of a horizontally scaled operator. Corresponds to the sharding set N ; joining or leaving the fleet is an instance of membership churn.

generational clock: A Lamport clock incremented on leader-election events. Every node stores the current generation; heartbeats or writes from an older generation are rejected, preventing a stale leader from acting after a new one has been elected. *Section 2.2* explains in more detail.

linearizability: Strongest single-object consistency model in which every operation appears to take effect atomically at some instant between its invocation and response, so that all clients observe a single, real-time-consistent order of operations [3].

membership churn: Changes in the set N caused by starting, stopping, updating or crashing nodes.

minimal disruption: When N changes, the number of keys whose assignment changes is close to the information-theoretic minimum $\frac{|K|}{|N|}$ [2].

monotonicity: Stricter property than minimal disruption: when N changes, keys are only reassigned to joining nodes or away from leaving nodes; no key ever moves between two nodes that are neither joining nor leaving [2].

K : Set of keys distributed over N

N : Set of member nodes available to process work

sharding: Partitioning a key set K into disjoint subsets and assigning each subset to one node of set N . This is achieved using a *sharding function* $f : K \rightarrow N$.

state machine replication: Design pattern that models a service as a deterministic state machine and replicates it across nodes by agreeing on a total order of inputs via a consensus protocol. Reduces distributed mutual exclusion to ordering agreement. *Section 2.2* explains in more detail.

Kubernetes Runtime

cluster: Throughout this thesis *cluster* refers to a Kubernetes cluster: a control plane together with kubelet-managed worker nodes, exposing a single Kubernetes API. Where a different meaning is required it is qualified explicitly (e.g. *etcd cluster* or *managed cluster*).

controller: Software component reconciling a specific Kubernetes *kind*. Observes the current cluster state, compares it against the desired state expressed by resources of its kind, and issues actions that drive the current state toward the desired state. *Section 2.3.1* explains in more detail.

controller-runtime: Go library maintained by the Kubernetes API Machinery Special Interest Group that provides the runtime abstractions (manager, cache, client, controller, work queue) on top of which Kubebuilder generates operator scaffolding.

etcd: Distributed, strongly consistent key-value store used as the backing store of a Kubernetes cluster. Exposes Key-Value, Watch, and Lease APIs and provides linearizable ordering via the Raft consensus algorithm. *Section 2.3.3* explains in more detail.

node: In the concrete Kubernetes sense, a worker machine running a kubelet and managed by the control plane. In the abstract sharding sense (N), a member of the set of instances over which work is distributed. The intended sense follows from context.

operator: A Kubernetes application that extends the cluster with domain-specific automation. In Kubebuilder terms, one operator consists of a single *manager* and one or more *controllers* reconciling CRs. *Section 2.4.1* explains in more detail.

reconciler: The developer-provided function inside a controller that implements the business logic of reconciling a single resource. Designed to be idempotent: it acts on the current state of a resource rather than on the event that triggered it. *Section 2.4.5* explains in more detail.

resource version: Monotonically increasing counter attached to every Kubernetes resource, surfaced from etcd's revision. Acts as a fencing token: modifying requests carrying an outdated resource version are rejected, and watch streams can resume from a specific resource version. *Section 2.3.2* explains in more detail.

shard: One replica of a horizontally scaled Kubernetes operator, responsible for reconciling a disjoint subset of resources assigned to it by a sharding function. *Section 3* surveys concrete shard implementations.

watch: Long-running Kubernetes API request that streams resource change events (added, modified, deleted) to a client without polling. The primary mechanism controllers use to detect state changes. *Section 2.3.2* explains in more detail.

worker: A controller may run one or more workers, each able to handle one reconcile request concurrently. Having n workers therefore allows n reconcile request to be handled concurrently.

List of Figures

Figure 1	Visualization of consistent hashing as described by Karger et al.	4
Figure 2	Three lookup states of DxHash: (a) initial four-node cluster, (b) after inserting node 4, (c) after removing node 1.	8
Figure 3	Sequence diagram showing the mechanism of a monotonically increasing fencing token.	11
Figure 4	Kubernetes architecture showing the relation between the control plane, Nodes and how a developer interacts with the cluster through the <i>kube-api-server</i> component.	16
Figure 5	Application Programming Interface (API) Server watch request fan out. .	20
Figure 6	Diagram showing the Kubebuilder architecture [1].	21
Figure 7	Relationship of client and cache interfaces in the controller-runtime library.	23
Figure 8	Data flow of a <code>Get</code> request against the controller-runtime cache. The informer is started lazily on the first request for a Group Version Kind (GVK) and keeps the local indexer in sync via a watch stream.	25
Figure 9	Architecture of the horizontal-scaling solution by Tim Ebert [48].	30
Figure 10	The generate/test cycle used in the search process according to Hevner et al. [51].	36
Figure 11	Class diagram showing the architecture of the <i>kube-sharder</i> library. . . .	45
Figure 12	Sequence diagram showing how the controller interacts with the wrapped work queue.	48
Figure 13	Class diagram showing the architecture of the sharded cache.	50
Figure 14	Revision control state machine.	55
Figure 15	Jain fairness index J for different shard and resource counts, computed from reconcile times over 60 s. Cells with fewer than two resources per shard are omitted, both because their fairness is expected to be low and to increase the color resolution of the remaining cells. Higher is better. . . .	61
Figure 16	Relative Resident Set Size (RSS) against resource count for different shard counts, with $\emptyset$ marking the unsharded baseline. Each run reports the median Resident Set Size (RSS) over the last 20 s of a 70 s run; points are medians across runs and whiskers show min/max. Left: unsharded cache; right: sharded cache. Lower is better.	62

- Figure 17 Relative CPU time against resource count for different shard counts, with $\emptyset$ marking the unsharded baseline. Measured over 60 s at a 10 s reconcile interval. Left: unsharded cache; right: sharded cache. Points are medians across runs; whiskers show min/max. Lower is better. 63
- Figure 18 Relative Central Processing Unit (CPU) time against per-reconcile compute cost for different shard counts, with the single-shard variant as baseline. Compute cost is simulated by computing SHA-256 hashes per reconcile. Measured over 60 s at a 10 s reconcile interval, sharded cache enabled. Points are medians across runs; whiskers show min/max. Lower is better. 64
- Figure 19 Relative API request count against resource count for different shard counts, with $\emptyset$ marking the unsharded baseline. Requests counted over 120 s while reconciling every website resource every 10 s with one status write per reconcile. Sharded cache enabled. Lower is better. 66
- Figure 20 Breakdown of API request types by shard count at 1000 resources. Same benchmark as *Figure 19*. 66
- Figure 21 Relative reconcile latency against shard count, with the unsharded variant as baseline (constant 1, omitted from the plot). Latency is the delay from an API event reaching the operator's informer to the reconcile function being invoked. Points are medians across runs; violins show the sample distribution with min/max extrema marked. Sharded cache enabled. Lower is better. 67
- Figure 22 Median failover speed-up against shard count, with the single-shard variant as baseline. Failover speed is measured as the time all shard work queues take to drain the reconcile burst triggered by replacing one operator pod. Points are medians across runs; whiskers show min/max. Benchmark: 1024 resources, each reconcile taking 1 s. Higher is better. . 68

List of Tables

Table I	Example assignment using Highest Random Weight (HRW). The highest weight per row determines the assigned node.	6
Table II	Verbs supported by the Kubernetes Application Programming Interface (API)	18
Table III	Comparison of the surveyed sharding approaches. A checkmark (✓) denotes that the property is fulfilled, a dash (—) denotes that it is not.	34
Table IV	Action matrix describing the required action for a resource depending on whether it is currently cached and whether it matches the predicate during cache resync.	53

List of Code

Code 1	Kubernetes resource example showing a resource of kind <i>Deployment</i> , defining a deployment of three pods that each run an <i>nginx</i> container and expose port 80.	14
Code 2	Kubernetes watch request example.	17
Code 3	Example of a reconcile function that requeues the reconciled resource after 10 minutes.	27
Code 4	Example of a <i>Website</i> resource, retrieving https://example.com every 10 seconds.	39

Statutory Declaration

I hereby declare that I have written my Bachelorarbeit (T3_3300) on the topic: “Extending Kubebuilder for Horizontal Kubernetes Operator Scaling” independently and have not used any sources or aids other than those specified.

Karlsruhe

Place

11.09.2026

Date

Marvin Fuchs

Signature